90 DSA Patterns

Java 17 Interview Handbook

Production-quality Java templates with signals, dry runs, pitfalls, and related LeetCode problems.

90 DSA Patterns - Java 17 Interview Handbook

Production-quality Java 17 conversion of the JavaScript DSA pattern set.

Audience: FAANG-style interviews, senior Java engineers, and DSA learners.

How to use: identify the signal, match keywords, recall the Java template, then adapt the invariant to the problem.

All code snippets use Java 17 collections such as ArrayList, HashMap, HashSet, Queue, Deque, PriorityQueue, TreeMap, and TreeSet where appropriate.

Shared tree node template used by tree snippets:

```
static final class TreeNode {
    int val; // Node value.
    TreeNode left; // Left child.
    TreeNode right; // Right child.
    TreeNode(int val) { this.val = val; } // Constructor.
}
```

Family: Sliding Window

Pattern 01: Fixed Size Window

1. Pattern Name

1. Fixed Size Window

2. Signal (when to recognize this pattern)

Find max, min, sum, or average over every contiguous block of exactly k elements.

3. Keywords

size k, fixed window, subarray, average, maximum sum

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: enumerate every contiguous candidate and recompute its state from scratch, usually O(n*k) or O(n^2).

Optimized approach: move each boundary only forward and update counts/sums incrementally.

The algorithm works because every valid contiguous window appears during the left/right sweep, and stale state is removed exactly when it leaves the window.

Edge cases: empty input, k <= 0, k larger than input, duplicated characters, and constraints that become invalid immediately.

Java notes: use HashMap.merge for frequency maps, int[26] for lowercase-only strings, and avoid substring creation in inner loops unless required.

```
public int maxSumFixedWindow(int[] nums, int k) {
    if (nums == null || k <= 0 || k > nums.length) return 0; // Guard invalid windows.
    int windowSum = 0; // Holds the sum of the current size-k window.
    for (int i = 0; i < k; i++) windowSum += nums[i]; // Build the first window once.
    int best = windowSum; // First complete window is the initial answer.
    for (int right = k; right < nums.length; right++) { // Slide one step at a time.
        windowSum += nums[right]; // Add the new element entering from the right.
        windowSum -= nums[right - k]; // Remove the element leaving from the left.
        best = Math.max(best, windowSum); // Keep the maximum window sum seen.
    }
    return best; // Return the optimized O(n) answer.
}
```

7. Dry Run Example

Dry run the template on Max sum subarray of size k: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'size k' and asks for find max, min, sum, or average over every contiguous block of exactly k elements., reach for Fixed Size Window before designing from scratch.

9. Common Mistakes

- Forgetting to remove the left element from every tracked structure.
- Updating the answer before the window is valid.
- Using substring inside the loop and accidentally making the solution O(n^2).

10. Related LeetCode Problems

LC 643 Maximum Average Subarray I; LC 1456 Maximum Vowels in a Substring

---

Pattern 02: Variable Size Window

1. Pattern Name

2. Variable Size Window

2. Signal (when to recognize this pattern)

Find the longest or shortest contiguous window that satisfies a changing constraint.

3. Keywords

longest, shortest, at most k, minimum length, substring

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: enumerate every contiguous candidate and recompute its state from scratch, usually O(n*k) or O(n^2).

Optimized approach: move each boundary only forward and update counts/sums incrementally.

The algorithm works because every valid contiguous window appears during the left/right sweep, and stale state is removed exactly when it leaves the window.

Edge cases: empty input, k <= 0, k larger than input, duplicated characters, and constraints that become invalid immediately.

Java notes: use HashMap.merge for frequency maps, int[26] for lowercase-only strings, and avoid substring creation in inner loops unless required.

```
public int lengthOfLongestSubstring(String s) {
    if (s == null || s.isEmpty()) return 0; // Empty input has length zero.
    Map<Character, Integer> lastSeen = new HashMap<>(); // Character -> latest index.
    int left = 0; // Left edge of the current valid window.
    int best = 0; // Best valid length found so far.
    for (int right = 0; right < s.length(); right++) { // Expand with each character.
        char c = s.charAt(right); // Current character entering the window.
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) { // Duplicate inside window.
            left = lastSeen.get(c) + 1; // Jump left after the previous copy.
        }
        lastSeen.put(c, right); // Record the latest position for c.
        best = Math.max(best, right - left + 1); // Update after restoring validity.
    }
    return best; // Longest substring with no repeated character.
}
```

7. Dry Run Example

Dry run the template on Longest substring without repeating characters: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'longest' and asks for find the longest or shortest contiguous window that satisfies a changing constraint., reach for Variable Size Window before designing from scratch.

9. Common Mistakes

- Forgetting to remove the left element from every tracked structure.
- Updating the answer before the window is valid.
- Using substring inside the loop and accidentally making the solution O(n^2).

10. Related LeetCode Problems

LC 3 Longest Substring Without Repeating Characters; LC 209 Minimum Size Subarray Sum

---

Pattern 03: String Permutation Window

1. Pattern Name

3. String Permutation Window

2. Signal (when to recognize this pattern)

Check whether a fixed-length window is an anagram or permutation of a pattern.

3. Keywords

permutation in string, anagram, contains, frequency

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: enumerate every contiguous candidate and recompute its state from scratch, usually O(n*k) or O(n^2).

Optimized approach: move each boundary only forward and update counts/sums incrementally.

The algorithm works because every valid contiguous window appears during the left/right sweep, and stale state is removed exactly when it leaves the window.

Edge cases: empty input, k <= 0, k larger than input, duplicated characters, and constraints that become invalid immediately.

Java notes: use HashMap.merge for frequency maps, int[26] for lowercase-only strings, and avoid substring creation in inner loops unless required.

```
public boolean checkInclusion(String pattern, String text) {
    if (pattern == null || text == null || pattern.length() > text.length()) return false; // Impossible case.
    int[] need = new int[26]; // Counts required lowercase letters.
    int[] window = new int[26]; // Counts letters in the current window.
    for (char c : pattern.toCharArray()) need[c - 'a']++; // Build target frequency.
    int k = pattern.length(); // Every candidate window must have this size.
    for (int right = 0; right < text.length(); right++) { // Move right over text.
        window[text.charAt(right) - 'a']++; // Add entering character.
        if (right >= k) window[text.charAt(right - k) - 'a']--; // Remove leaving character.
        if (Arrays.equals(need, window)) return true; // Same counts means permutation.
    }
    return false; // No window matched the pattern counts.
}
```

7. Dry Run Example

Dry run the template on Permutation in String: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'permutation in string' and asks for check whether a fixed-length window is an anagram or permutation of a pattern., reach for String Permutation Window before designing from scratch.

9. Common Mistakes

- Forgetting to remove the left element from every tracked structure.
- Updating the answer before the window is valid.
- Using substring inside the loop and accidentally making the solution O(n^2).

10. Related LeetCode Problems

LC 567 Permutation in String; LC 438 Find All Anagrams in a String

---

Pattern 04: Window with HashMap

1. Pattern Name

4. Window with HashMap

2. Signal (when to recognize this pattern)

Find the smallest substring/window containing all required characters or tokens.

3. Keywords

minimum window, contains all, substring, frequency map

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: enumerate every contiguous candidate and recompute its state from scratch, usually O(n*k) or O(n^2).

Optimized approach: move each boundary only forward and update counts/sums incrementally.

The algorithm works because every valid contiguous window appears during the left/right sweep, and stale state is removed exactly when it leaves the window.

Edge cases: empty input, k <= 0, k larger than input, duplicated characters, and constraints that become invalid immediately.

Java notes: use HashMap.merge for frequency maps, int[26] for lowercase-only strings, and avoid substring creation in inner loops unless required.

```
public String minWindow(String s, String t) {
    if (s == null || t == null || s.length() < t.length()) return ""; // No valid window possible.
    Map<Character, Integer> need = new HashMap<>(); // Required counts by character.
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum); // Build target map.
    Map<Character, Integer> window = new HashMap<>(); // Current window counts.
    int required = need.size(); // Number of character types to satisfy.
    int formed = 0, left = 0, bestLen = Integer.MAX_VALUE, bestStart = 0; // Window state.
    for (int right = 0; right < s.length(); right++) { // Expand right boundary.
        char c = s.charAt(right); // Character entering the window.
        window.merge(c, 1, Integer::sum); // Count the entering character.
        if (need.containsKey(c) && window.get(c).equals(need.get(c))) formed++; // Type satisfied.
        while (formed == required) { // Shrink while all requirements are met.
            if (right - left + 1 < bestLen) { bestLen = right - left + 1; bestStart = left; } // Save best.
            char remove = s.charAt(left++); // Character leaving from the left.
            window.merge(remove, -1, Integer::sum); // Decrease its window count.
            if (need.containsKey(remove) && window.get(remove) < need.get(remove)) formed--; // Lost validity.
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen); // Answer.
}
```

7. Dry Run Example

Dry run the template on Minimum Window Substring: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'minimum window' and asks for find the smallest substring/window containing all required characters or tokens., reach for Window with HashMap before designing from scratch.

9. Common Mistakes

- Forgetting to remove the left element from every tracked structure.
- Updating the answer before the window is valid.
- Using substring inside the loop and accidentally making the solution O(n^2).

10. Related LeetCode Problems

LC 76 Minimum Window Substring; LC 727 Minimum Window Subsequence

---

Pattern 05: Multi-pointer Window

1. Pattern Name

5. Multi-pointer Window

2. Signal (when to recognize this pattern)

Maintain a variable window while a count of distinct elements stays at most k.

3. Keywords

at most k distinct, fruit baskets, frequency window

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: enumerate every contiguous candidate and recompute its state from scratch, usually O(n*k) or O(n^2).

Optimized approach: move each boundary only forward and update counts/sums incrementally.

The algorithm works because every valid contiguous window appears during the left/right sweep, and stale state is removed exactly when it leaves the window.

Edge cases: empty input, k <= 0, k larger than input, duplicated characters, and constraints that become invalid immediately.

Java notes: use HashMap.merge for frequency maps, int[26] for lowercase-only strings, and avoid substring creation in inner loops unless required.

```
public int longestAtMostKDistinct(int[] nums, int k) {
    if (nums == null || k <= 0) return 0; // No positive distinct budget.
    Map<Integer, Integer> freq = new HashMap<>(); // Value -> count inside window.
    int left = 0, best = 0; // Window boundary and best length.
    for (int right = 0; right < nums.length; right++) { // Expand right.
        freq.merge(nums[right], 1, Integer::sum); // Add right value.
        while (freq.size() > k) { // Too many distinct values.
            int value = nums[left++]; // Remove left value and move boundary.
            freq.merge(value, -1, Integer::sum); // Decrement count.
            if (freq.get(value) == 0) freq.remove(value); // Remove absent value.
        }
        best = Math.max(best, right - left + 1); // Window is valid here.
    }
    return best; // Longest valid window length.
}
```

7. Dry Run Example

Dry run the template on Fruit Into Baskets: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'at most k distinct' and asks for maintain a variable window while a count of distinct elements stays at most k., reach for Multi-pointer Window before designing from scratch.

9. Common Mistakes

- Forgetting to remove the left element from every tracked structure.
- Updating the answer before the window is valid.
- Using substring inside the loop and accidentally making the solution O(n^2).

10. Related LeetCode Problems

LC 904 Fruit Into Baskets; LC 340 Longest Substring with At Most K Distinct Characters

---

Family: Two Pointers

Pattern 06: Converging Pointers

1. Pattern Name

6. Converging Pointers

2. Signal (when to recognize this pattern)

Find a pair in a sorted array by moving inward from both ends.

3. Keywords

sorted array, pair sum, two sum sorted

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
public int[] twoSumSorted(int[] nums, int target) {
    int left = 0; // Smallest candidate index.
    int right = nums.length - 1; // Largest candidate index.
    while (left < right) { // Stop when candidates cross.
        int sum = nums[left] + nums[right]; // Current pair sum.
        if (sum == target) return new int[] {left, right}; // Found answer.
        if (sum < target) left++; // Need a larger sum, so move left rightward.
        else right--; // Need a smaller sum, so move right leftward.
    }
    return new int[] {-1, -1}; // Pair does not exist.
}
```

7. Dry Run Example

Dry run the template on Two Sum II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'sorted array' and asks for find a pair in a sorted array by moving inward from both ends., reach for Converging Pointers before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 167 Two Sum II; LC 125 Valid Palindrome

Pattern 07: Fast and Slow Pointers

1. Pattern Name

7. Fast and Slow Pointers

2. Signal (when to recognize this pattern)

Detect cycles or locate middle nodes using pointers moving at different speeds.

3. Keywords

cycle, linked list, middle, loop, tortoise hare

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
static final class ListNode {
    int val; // Stored value.
    ListNode next; // Pointer to the next node.
    ListNode(int val) { this.val = val; } // Simple constructor.
}

public boolean hasLinkedListCycle(ListNode head) {
    ListNode slow = head; // Moves one step.
    ListNode fast = head; // Moves two steps.
    while (fast != null && fast.next != null) { // Fast must be able to advance.
        slow = slow.next; // One-hop move.
        fast = fast.next.next; // Two-hop move.
        if (slow == fast) return true; // Meeting means a cycle exists.
    }
    return false; // Fast reached null, so no cycle.
}
```

7. Dry Run Example

Dry run the template on Linked List Cycle: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'cycle' and asks for detect cycles or locate middle nodes using pointers moving at different speeds., reach for Fast and Slow Pointers before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 141 Linked List Cycle; LC 876 Middle of the Linked List

Pattern 08: Partition Pointers

1. Pattern Name

8. Partition Pointers

2. Signal (when to recognize this pattern)

Rearrange values in place around categories or a pivot.

3. Keywords

sort colors, Dutch flag, partition, 0 1 2

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
public void sortColors(int[] nums) {
    int low = 0; // Boundary after last 0.
    int mid = 0; // Current scanner.
    int high = nums.length - 1; // Boundary before first 2.
    while (mid <= high) { // Unknown region is [mid, high].
        if (nums[mid] == 0) { // 0 belongs on the left.
            swap(nums, low++, mid++); // Place 0 and advance both boundaries.
        } else if (nums[mid] == 1) { // 1 belongs in the middle.
            mid++; // Already placed correctly.
        } else { // nums[mid] == 2 belongs on the right.
            swap(nums, mid, high--); // Recheck mid because swapped value is unknown.
        }
    }
}

private void swap(int[] nums, int i, int j) {
    int tmp = nums[i]; nums[i] = nums[j]; nums[j] = tmp; // Standard in-place swap.
}
```

7. Dry Run Example

Dry run the template on Sort Colors: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'sort colors' and asks for rearrange values in place around categories or a pivot., reach for Partition Pointers before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 75 Sort Colors; LC 283 Move Zeroes

Pattern 09: Three Pointers - 3Sum

1. Pattern Name

9. Three Pointers - 3Sum

2. Signal (when to recognize this pattern)

After sorting, fix one value and solve a two-sum window for the remaining two.

3. Keywords

3Sum, triplet, three numbers, duplicates

4. Time Complexity

O(n^2)

5. Space Complexity

O(1) excluding output

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums); // Sorting enables two pointers and duplicate skipping.
    List<List<Integer>> ans = new ArrayList<>(); // Stores unique triplets.
    for (int i = 0; i < nums.length - 2; i++) { // Fix the first value.
        if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip duplicate first values.
        int left = i + 1, right = nums.length - 1; // Search remaining pair.
        while (left < right) { // Standard sorted two-sum scan.
            int sum = nums[i] + nums[left] + nums[right]; // Current triplet sum.
            if (sum == 0) { // Found a valid triplet.
                ans.add(List.of(nums[i], nums[left], nums[right])); // Save immutable triplet.
                while (left < right && nums[left] == nums[left + 1]) left++; // Skip duplicate left.
                while (left < right && nums[right] == nums[right - 1]) right--; // Skip duplicate right.
                left++; right--; // Move to next distinct pair.
            } else if (sum < 0) left++; // Need larger sum.
            else right--; // Need smaller sum.
        }
    }
    return ans; // All unique zero-sum triplets.
}
```

7. Dry Run Example

Dry run the template on 3Sum: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says '3Sum' and asks for after sorting, fix one value and solve a two-sum window for the remaining two., reach for Three Pointers - 3Sum before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 15 3Sum; LC 16 3Sum Closest

Pattern 10: Merge Pointers

1. Pattern Name

10. Merge Pointers

2. Signal (when to recognize this pattern)

Merge two sorted inputs by advancing the pointer with the smaller current value.

3. Keywords

merge sorted, two sorted arrays, two sorted lists

4. Time Complexity

O(n + m)

5. Space Complexity

O(n + m)

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
public int[] mergeSortedArrays(int[] a, int[] b) {
    int[] merged = new int[a.length + b.length]; // Output has all elements.
    int i = 0, j = 0, write = 0; // Pointers for a, b, and output.
    while (i < a.length && j < b.length) { // Merge while both arrays have values.
        if (a[i] <= b[j]) merged[write++] = a[i++]; // Take smaller from a.
        else merged[write++] = b[j++]; // Take smaller from b.
    }
    while (i < a.length) merged[write++] = a[i++]; // Copy remaining a values.
    while (j < b.length) merged[write++] = b[j++]; // Copy remaining b values.
    return merged; // Sorted combined array.
}
```

7. Dry Run Example

Dry run the template on Merge two sorted arrays: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'merge sorted' and asks for merge two sorted inputs by advancing the pointer with the smaller current value., reach for Merge Pointers before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 88 Merge Sorted Array; LC 21 Merge Two Sorted Lists

Pattern 11: Remove Duplicates

1. Pattern Name

11. Remove Duplicates

2. Signal (when to recognize this pattern)

Compress a sorted array in place by writing each new value once.

3. Keywords

remove duplicates, in-place, sorted, slow fast

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try all pairs/triples or copy into extra arrays, which usually costs O(n^2) or worse.

Optimized approach: exploit sorted order or in-place partition invariants so each pointer advances monotonically.

The algorithm works because each pointer move discards candidates that cannot improve or satisfy the answer under the maintained invariant.

Edge cases: duplicates, arrays of length 0 or 1, negative numbers, all equal values, and index return convention.

Java notes: use Arrays.sort for 3Sum, return int[] for index pairs, and use helper swap methods for clean in-place code.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0; // Empty array has new length zero.
    int write = 1; // Next position for a unique value.
    for (int read = 1; read < nums.length; read++) { // Scan remaining values.
        if (nums[read] != nums[write - 1]) { // New unique value found.
            nums[write++] = nums[read]; // Write it into compressed prefix.
        }
    }
    return write; // Values in nums[0..write-1] are unique.
}
```

7. Dry Run Example

Dry run the template on Remove Duplicates from Sorted Array: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'remove duplicates' and asks for compress a sorted array in place by writing each new value once., reach for Remove Duplicates before designing from scratch.

9. Common Mistakes

- Moving both pointers when only one invariant justifies it.
- Forgetting to skip duplicates in 3Sum.
- Returning one-based indices when the interviewer expects zero-based indices.

10. Related LeetCode Problems

LC 26 Remove Duplicates from Sorted Array; LC 80 Remove Duplicates II

Family: Binary Search

Pattern 12: Classic Binary Search

1. Pattern Name

12. Classic Binary Search

2. Signal (when to recognize this pattern)

Search a sorted monotonic range for a target value.

3. Keywords

sorted array, search, find target, log n

4. Time Complexity

O(log n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public int binarySearch(int[] nums, int target) {
    int left = 0; // Inclusive left boundary.
    int right = nums.length - 1; // Inclusive right boundary.
    while (left <= right) { // Search space is non-empty.
        int mid = left + (right - left) / 2; // Overflow-safe midpoint.
        if (nums[mid] == target) return mid; // Target found.
        if (nums[mid] < target) left = mid + 1; // Discard left half.
        else right = mid - 1; // Discard right half.
    }
    return -1; // Target not present.
}
```

7. Dry Run Example

Dry run the template on Binary Search: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'sorted array' and asks for search a sorted monotonic range for a target value., reach for Classic Binary Search before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 704 Binary Search; LC 35 Search Insert Position

Pattern 13: Search Rotated Array

1. Pattern Name

13. Search Rotated Array

2. Signal (when to recognize this pattern)

Binary search a sorted array that has been rotated around a pivot.

3. Keywords

rotated, pivot, sorted rotated

4. Time Complexity

O(log n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public int searchRotated(int[] nums, int target) {
    int left = 0, right = nums.length - 1; // Current search range.
    while (left <= right) { // Continue while range is valid.
        int mid = left + (right - left) / 2; // Overflow-safe midpoint.
        if (nums[mid] == target) return mid; // Found target.
        if (nums[left] <= nums[mid]) { // Left half is sorted.
            if (nums[left] <= target && target < nums[mid]) right = mid - 1; // Target in left half.
            else left = mid + 1; // Target in right half.
        } else { // Right half is sorted.
            if (nums[mid] < target && target <= nums[right]) left = mid + 1; // Target in right half.
            else right = mid - 1; // Target in left half.
        }
    }
    return -1; // Target not found.
}
```

7. Dry Run Example

Dry run the template on Search in Rotated Sorted Array: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'rotated' and asks for binary search a sorted array that has been rotated around a pivot., reach for Search Rotated Array before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 33 Search in Rotated Sorted Array; LC 81 Search in Rotated Sorted Array II

Pattern 14: Find Boundary

1. Pattern Name

14. Find Boundary

2. Signal (when to recognize this pattern)

Find the first or last index where a monotonic predicate becomes true.

3. Keywords

first position, last occurrence, leftmost, rightmost

4. Time Complexity

O(log n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1, ans = -1; // Track best index found.
    while (left <= right) { // Binary search range.
        int mid = left + (right - left) / 2; // Safe midpoint.
        if (nums[mid] >= target) { // Boundary could be at mid or left.
            if (nums[mid] == target) ans = mid; // Save candidate first occurrence.
            right = mid - 1; // Continue left.
        } else {
            left = mid + 1; // Need larger values.
        }
    }
    return ans; // -1 if target never appeared.
}
```

7. Dry Run Example

Dry run the template on Find first and last position: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'first position' and asks for find the first or last index where a monotonic predicate becomes true., reach for Find Boundary before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 34 Find First and Last Position; LC 278 First Bad Version

Pattern 15: Binary Search on Answer

1. Pattern Name

15. Binary Search on Answer

2. Signal (when to recognize this pattern)

The answer is numeric and feasibility is monotonic as the answer grows or shrinks.

3. Keywords

minimize maximum, maximize minimum, capacity, Koko, split array

4. Time Complexity

O(n log R)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public int shipWithinDays(int[] weights, int days) {
    int left = 0, right = 0; // Search space for capacity.
    for (int w : weights) { left = Math.max(left, w); right += w; } // Min and max capacity.
    while (left < right) { // Find smallest feasible capacity.
        int mid = left + (right - left) / 2; // Candidate capacity.
        if (canShip(weights, days, mid)) right = mid; // Feasible, try smaller.
        else left = mid + 1; // Infeasible, need larger capacity.
    }
    return left; // Minimum capacity that works.
}

private boolean canShip(int[] weights, int days, int capacity) {
    int usedDays = 1, load = 0; // Current simulation state.
    for (int w : weights) { // Process packages in order.
        if (load + w > capacity) { usedDays++; load = 0; } // Start a new day.
        load += w; // Put package on current day.
    }
    return usedDays <= days; // Monotonic feasibility predicate.
}
```

7. Dry Run Example

Dry run the template on Capacity to Ship Packages: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'minimize maximum' and asks for the answer is numeric and feasibility is monotonic as the answer grows or shrinks., reach for Binary Search on Answer before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 875 Koko Eating Bananas; LC 410 Split Array Largest Sum; LC 1011 Capacity To Ship Packages

---

Pattern 16: Peak Finding

1. Pattern Name

16. Peak Finding

2. Signal (when to recognize this pattern)

Find a local maximum by following the slope in a mountain-like search space.

3. Keywords

peak, mountain array, find peak, slope

4. Time Complexity

O(log n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public int findPeakElement(int[] nums) {
    int left = 0, right = nums.length - 1; // Peak must exist in this range.
    while (left < right) { // Stop when one index remains.
        int mid = left + (right - left) / 2; // Safe midpoint.
        if (nums[mid] > nums[mid + 1]) right = mid; // Downward slope means peak is left.
        else left = mid + 1; // Upward slope means peak is right.
    }
    return left; // Remaining index is a peak.
}
```

7. Dry Run Example

Dry run the template on Find Peak Element: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'peak' and asks for find a local maximum by following the slope in a mountain-like search space., reach for Peak Finding before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 162 Find Peak Element; LC 852 Peak Index in a Mountain Array

---

Pattern 17: Search 2D Matrix

1. Pattern Name

17. Search 2D Matrix

2. Signal (when to recognize this pattern)

Treat a sorted matrix as one virtual sorted array.

3. Keywords

matrix search, 2D sorted, row column

4. Time Complexity

O(log(mn))

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
public boolean searchMatrix(int[][] matrix, int target) {
    if (matrix.length == 0 || matrix[0].length == 0) return false; // Empty matrix guard.
    int rows = matrix.length, cols = matrix[0].length; // Dimensions.
    int left = 0, right = rows * cols - 1; // Virtual 1D range.
    while (left <= right) { // Standard binary search.
        int mid = left + (right - left) / 2; // Virtual midpoint.
        int value = matrix[mid / cols][mid % cols]; // Map 1D index to row and col.
        if (value == target) return true; // Found target.
        if (value < target) left = mid + 1; // Discard lower half.
        else right = mid - 1; // Discard upper half.
    }
    return false; // Target absent.
}
```

7. Dry Run Example

Dry run the template on Search a 2D Matrix: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'matrix search' and asks for treat a sorted matrix as one virtual sorted array., reach for Search 2D Matrix before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 74 Search a 2D Matrix; LC 240 Search a 2D Matrix II

Pattern 18: Infinite Array Search

1. Pattern Name

18. Infinite Array Search

2. Signal (when to recognize this pattern)

The right boundary is unknown, so expand exponentially before binary search.

3. Keywords

infinite array, unknown size, unbounded, reader

4. Time Complexity

O(log n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: scan the range linearly or try every answer, costing O(n) to O(nR).

Optimized approach: identify a monotonic search space and discard half with each midpoint check.

The algorithm works because the predicate or sorted order divides the search space into impossible and possible halves.

Edge cases: empty input, duplicate values, overflow in midpoint calculation, rotated boundaries, and off-by-one termination.

Java notes: always compute mid as left + (right - left) / 2 and be explicit about inclusive versus exclusive boundaries.

- Binary search search space: define low/high so the answer is guaranteed inside; shrink toward the first feasible or exact target.
- Mid calculation: use left + (right - left) / 2 to avoid overflow.

```
interface ArrayReader {
    int get(int index); // Returns a large sentinel when index is out of bounds.
}

public int searchUnknownSize(ArrayReader reader, int target) {
    int left = 0, right = 1; // Start with a tiny known range.
    while (reader.get(right) < target) { // Expand until target could fit.
        left = right + 1; // Previous right is too small.
        right *= 2; // Exponential growth keeps O(log n).
    }
    while (left <= right) { // Binary search inside discovered range.
        int mid = left + (right - left) / 2; // Safe midpoint.
        int value = reader.get(mid); // Read candidate value.
        if (value == target) return mid; // Found target.
        if (value < target) left = mid + 1; // Need larger values.
        else right = mid - 1; // Need smaller values.
    }
    return -1; // Target not found.
}
```

7. Dry Run Example

Dry run the template on Search in unknown-size sorted array: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'infinite array' and asks for the right boundary is unknown, so expand exponentially before binary search., reach for Infinite Array Search before designing from scratch.

9. Common Mistakes

- Using (left + right) / 2 and risking overflow.
- Mixing inclusive and exclusive boundaries.
- Not proving the feasibility predicate is monotonic.

10. Related LeetCode Problems

LC 702 Search in a Sorted Array of Unknown Size

Family: Prefix Sum

Pattern 19: 1D Prefix Sum

1. Pattern Name

19. 1D Prefix Sum

2. Signal (when to recognize this pattern)

Answer many range-sum queries after one linear preprocessing pass.

3. Keywords

range sum, subarray sum, immutable query

4. Time Complexity

O(n) build, O(1) query

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: sum each range or apply each update element-by-element, often O(nq).

Optimized approach: precompute cumulative state so a range becomes subtraction, or defer updates with differences.

The algorithm works because prefix sums encode all previous values, and subtracting shared prefixes isolates exactly the requested range.

Edge cases: ranges starting at zero, negative values, large sums requiring long, and empty matrices.

Java notes: prefer long[] or long[][] when sums can overflow int; use one-based prefix arrays to simplify boundaries.

```
static final class PrefixSum1D {
    private final long[] prefix; // prefix[i] stores sum of nums[0..i-1].

    PrefixSum1D(int[] nums) {
        prefix = new long[nums.length + 1]; // Extra zero handles ranges starting at 0.
        for (int i = 0; i < nums.length; i++) prefix[i + 1] = prefix[i] + nums[i]; // Build cumulatively.
    }

    long rangeSum(int left, int right) {
        return prefix[right + 1] - prefix[left]; // Inclusive range [left, right].
    }
}
```

7. Dry Run Example

Dry run the template on Range Sum Query Immutable: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'range sum' and asks for answer many range-sum queries after one linear preprocessing pass., reach for 1D Prefix Sum before designing from scratch.

9. Common Mistakes

- Missing prefix[0] = 0.
- Using int when range sums can exceed 32-bit.
- Getting inclusive range endpoints off by one.

10. Related LeetCode Problems

LC 303 Range Sum Query Immutable; LC 304 Range Sum Query 2D

Pattern 20: Prefix Sum + HashMap

1. Pattern Name

20. Prefix Sum + HashMap

2. Signal (when to recognize this pattern)

Count subarrays by storing how many previous prefix sums would complete target k.

3. Keywords

subarray sum equals k, count subarrays, prefix map

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: sum each range or apply each update element-by-element, often O(nq).

Optimized approach: precompute cumulative state so a range becomes subtraction, or defer updates with differences.

The algorithm works because prefix sums encode all previous values, and subtracting shared prefixes isolates exactly the requested range.

Edge cases: ranges starting at zero, negative values, large sums requiring long, and empty matrices.

Java notes: prefer long[] or long[][] when sums can overflow int; use one-based prefix arrays to simplify boundaries.

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> countByPrefix = new HashMap<>(); // Prefix sum -> frequency.
    countByPrefix.put(0, 1); // Empty prefix allows subarrays starting at index 0.
    int prefix = 0, count = 0; // Running sum and answer.
    for (int value : nums) { // Extend subarray end one value at a time.
        prefix += value; // Current prefix sum.
        count += countByPrefix.getOrDefault(prefix - k, 0); // Previous prefixes that form k.
        countByPrefix.merge(prefix, 1, Integer::sum); // Store current prefix.
    }
    return count; // Number of subarrays summing to k.
}
```

7. Dry Run Example

Dry run the template on Subarray Sum Equals K: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'subarray sum equals k' and asks for count subarrays by storing how many previous prefix sums would complete target k., reach for Prefix Sum + HashMap before designing from scratch.

9. Common Mistakes

- Missing prefix[0] = 0.
- Using int when range sums can exceed 32-bit.
- Getting inclusive range endpoints off by one.

10. Related LeetCode Problems

LC 560 Subarray Sum Equals K; LC 974 Subarray Sums Divisible by K

Pattern 21: 2D Prefix Sum

1. Pattern Name

21. 2D Prefix Sum

2. Signal (when to recognize this pattern)

Answer rectangle-sum queries in constant time after matrix preprocessing.

3. Keywords

matrix sum, rectangle sum, 2D query

4. Time Complexity

O(mn) build, O(1) query

5. Space Complexity

O(mn)

6. Java 17 Template

Brute force: sum each range or apply each update element-by-element, often O(nq).

Optimized approach: precompute cumulative state so a range becomes subtraction, or defer updates with differences.

The algorithm works because prefix sums encode all previous values, and subtracting shared prefixes isolates exactly the requested range.

Edge cases: ranges starting at zero, negative values, large sums requiring long, and empty matrices.

Java notes: prefer long[] or long[][] when sums can overflow int; use one-based prefix arrays to simplify boundaries.

```
static final class PrefixSum2D {
    private final long[][] prefix; // One-based rectangle prefix sums.

    PrefixSum2D(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length; // Matrix dimensions.
        prefix = new long[rows + 1][cols + 1]; // Extra row and col remove boundary checks.
        for (int r = 1; r <= rows; r++) { // Build each prefix cell.
            for (int c = 1; c <= cols; c++) { // One-based column index.
                prefix[r][c] = matrix[r - 1][c - 1] + prefix[r - 1][c] + prefix[r][c - 1] - prefix[r - 1][c -
1]; // Inclusion-exclusion.
            }
        }
    }

    long sumRegion(int r1, int c1, int r2, int c2) {
        return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1] - prefix[r2 + 1][c1] + prefix[r1][c1]; // Rectangle
sum.
    }
}
```

7. Dry Run Example

Dry run the template on Range Sum Query 2D Immutable: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'matrix sum' and asks for answer rectangle-sum queries in constant time after matrix preprocessing., reach for 2D Prefix Sum before designing from scratch.

9. Common Mistakes

- Missing prefix[0] = 0.
- Using int when range sums can exceed 32-bit.
- Getting inclusive range endpoints off by one.

10. Related LeetCode Problems

LC 304 Range Sum Query 2D Immutable; LC 1314 Matrix Block Sum

---

Pattern 22: Running Difference Array

1. Pattern Name

22. Running Difference Array

2. Signal (when to recognize this pattern)

Apply many range updates lazily, then materialize values with a running sum.

3. Keywords

range update, difference array, car pooling, bookings

4. Time Complexity

O(n + q)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: sum each range or apply each update element-by-element, often O(nq).

Optimized approach: precompute cumulative state so a range becomes subtraction, or defer updates with differences.

The algorithm works because prefix sums encode all previous values, and subtracting shared prefixes isolates exactly the requested range.

Edge cases: ranges starting at zero, negative values, large sums requiring long, and empty matrices.

Java notes: prefer long[] or long[][] when sums can overflow int; use one-based prefix arrays to simplify boundaries.

```
public int[] applyRangeUpdates(int n, int[][] updates) {
    int[] diff = new int[n + 1]; // Difference array with sentinel at n.
    for (int[] update : updates) { // update = [left, right, delta].
        int left = update[0], right = update[1], delta = update[2]; // Unpack update.
        diff[left] += delta; // Start applying delta at left.
        if (right + 1 < diff.length) diff[right + 1] -= delta; // Stop after right.
    }
    int[] result = new int[n]; // Final materialized values.
    int running = 0; // Active accumulated delta.
    for (int i = 0; i < n; i++) { // Rebuild actual array.
        running += diff[i]; // Add changes starting or ending here.
        result[i] = running; // Store final value.
    }
    return result; // Values after all range updates.
}
```

7. Dry Run Example

Dry run the template on Corporate Flight Bookings: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'range update' and asks for apply many range updates lazily, then materialize values with a running sum., reach for Running Difference Array before designing from scratch.

9. Common Mistakes

- Missing prefix[0] = 0.
- Using int when range sums can exceed 32-bit.
- Getting inclusive range endpoints off by one.

10. Related LeetCode Problems

LC 1109 Corporate Flight Bookings; LC 1094 Car Pooling

Family: HashMap / HashSet

Pattern 23: Two Sum Pattern

1. Pattern Name

23. Two Sum Pattern

2. Signal (when to recognize this pattern)

Use a HashMap to remember seen values and locate complements in one pass.

3. Keywords

two sum, complement, pair, target

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> indexByValue = new HashMap<>(); // Seen value -> index.
    for (int i = 0; i < nums.length; i++) { // Scan each value once.
        int complement = target - nums[i]; // Value needed to complete target.
        if (indexByValue.containsKey(complement)) { // Complement was seen earlier.
            return new int[] {indexByValue.get(complement), i}; // Return pair indices.
        }
        indexByValue.put(nums[i], i); // Store current value after lookup.
    }
    return new int[0]; // No valid pair.
}
```

7. Dry Run Example

Dry run the template on Two Sum: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'two sum' and asks for use a hashmap to remember seen values and locate complements in one pass., reach for Two Sum Pattern before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 1 Two Sum; LC 653 Two Sum IV

Pattern 24: Frequency Counter

1. Pattern Name

24. Frequency Counter

2. Signal (when to recognize this pattern)

Compare counts of characters or values instead of sorting every time.

3. Keywords

anagram, frequency, count, valid anagram

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false; // Different lengths cannot match.
    int[] freq = new int[26]; // Lowercase character counts.
    for (char c : s.toCharArray()) freq[c - 'a']++; // Count first string.
    for (char c : t.toCharArray()) { // Remove counts using second string.
        if (--freq[c - 'a'] < 0) return false; // More of c in t than s.
    }
    return true; // All counts balanced.
}
```

7. Dry Run Example

Dry run the template on Valid Anagram: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'anagram' and asks for compare counts of characters or values instead of sorting every time., reach for Frequency Counter before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 242 Valid Anagram; LC 383 Ransom Note

Pattern 25: Group by Key

1. Pattern Name

25. Group by Key

2. Signal (when to recognize this pattern)

Normalize each item into a canonical key and group values sharing that key.

3. Keywords

group anagrams, categorize, bucket, canonical key

4. Time Complexity

O(n * k log k)

5. Space Complexity

O(n * k)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> groups = new HashMap<>(); // Key -> words with same key.
    for (String s : strs) { // Process each word.
        char[] chars = s.toCharArray(); // Convert to sortable chars.
        Arrays.sort(chars); // Sorted letters form canonical key.
        String key = new String(chars); // Build immutable map key.
        groups.computeIfAbsent(key, ignored -> new ArrayList<>()).add(s); // Add to group.
    }
    return new ArrayList<>(groups.values()); // Return grouped anagrams.
}
```

7. Dry Run Example

Dry run the template on Group Anagrams: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'group anagrams' and asks for normalize each item into a canonical key and group values sharing that key., reach for Group by Key before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 49 Group Anagrams; LC 249 Group Shifted Strings

Pattern 26: LRU Cache

1. Pattern Name

26. LRU Cache

2. Signal (when to recognize this pattern)

Combine O(1) key lookup with recency ordering and eviction.

3. Keywords

LRU, cache, evict, capacity, least recent

4. Time Complexity

O(1)

5. Space Complexity

O(capacity)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
static final class LRUCache extends LinkedHashMap<Integer, Integer> {
    private final int capacity; // Maximum number of entries.

    LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder=true moves reads/writes to the end.
        this.capacity = capacity; // Save capacity for eviction.
    }

    int getValue(int key) {
        return getOrDefault(key, -1); // LinkedHashMap updates recency on get.
    }

    void putValue(int key, int value) {
        put(key, value); // LinkedHashMap updates recency on put.
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
        return size() > capacity; // Evict least recently used entry automatically.
    }
}
```

7. Dry Run Example

Dry run the template on LRU Cache: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'LRU' and asks for combine o(1) key lookup with recency ordering and eviction., reach for LRU Cache before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 146 LRU Cache; LC 460 LFU Cache

Pattern 27: Complement Lookup

1. Pattern Name

27. Complement Lookup

2. Signal (when to recognize this pattern)

Precompute pair sums or differences so later pairs can find complements quickly.

3. Keywords

four sum, count pairs, complement, pair sums

4. Time Complexity

O(n^2)

5. Space Complexity

O(n^2)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
public int fourSumCount(int[] a, int[] b, int[] c, int[] d) {
    Map<Integer, Integer> pairSums = new HashMap<>(); // Sum of a+b -> count.
    for (int x : a) for (int y : b) pairSums.merge(x + y, 1, Integer::sum); // Precompute left pairs.
    int count = 0; // Number of zero-sum tuples.
    for (int x : c) { // Pick value from c.
        for (int y : d) { // Pick value from d.
            count += pairSums.getOrDefault(-(x + y), 0); // Need opposite pair sum.
        }
    }
    return count; // Total tuples.
}
```

7. Dry Run Example

Dry run the template on 4Sum II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'four sum' and asks for precompute pair sums or differences so later pairs can find complements quickly., reach for Complement Lookup before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 454 4Sum II; LC 18 4Sum

Pattern 28: Sliding Window + Map

1. Pattern Name

28. Sliding Window + Map

2. Signal (when to recognize this pattern)

Track frequency or distinct counts inside a sliding window.

3. Keywords

k distinct, frequency window, characters, counts

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: nested loops or repeated sorting/checking for each candidate.

Optimized approach: store previously computed facts in HashMap or HashSet for O(1) average lookup.

The algorithm works because the needed complement, key, or frequency can be represented as a deterministic lookup.

Edge cases: duplicate keys, missing values, negative numbers, Unicode strings, and capacity zero caches.

Java notes: choose HashMap for average O(1), TreeMap when ordering matters, and LinkedHashMap for LRU access order.

```
public int longestKDistinct(String s, int k) {
    if (k == 0) return 0; // No characters allowed.
```

```
    Map<Character, Integer> freq = new HashMap<>(); // Counts inside window.
    int left = 0, best = 0; // Window start and best length.
    for (int right = 0; right < s.length(); right++) { // Expand window.
        freq.merge(s.charAt(right), 1, Integer::sum); // Add right char.
        while (freq.size() > k) { // Too many distinct chars.
            char out = s.charAt(left++); // Remove left char.
            freq.merge(out, -1, Integer::sum); // Decrease count.
            if (freq.get(out) == 0) freq.remove(out); // Keep map size accurate.
        }
        best = Math.max(best, right - left + 1); // Window is valid.
    }
    return best; // Longest valid substring length.
}
```

7. Dry Run Example

Dry run the template on Longest substring with at most k distinct: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'k distinct' and asks for track frequency or distinct counts inside a sliding window., reach for Sliding Window + Map before designing from scratch.

9. Common Mistakes

- Overwriting an index before checking its complement.
- Using mutable objects as keys.
- Forgetting that HashMap has no deterministic iteration order.

10. Related LeetCode Problems

LC 340 Longest Substring with At Most K Distinct; LC 159 Longest Substring with At Most Two Distinct

---

Family: Stack

Pattern 29: Valid Parentheses

1. Pattern Name

29. Valid Parentheses

2. Signal (when to recognize this pattern)

Match nested openings with the most recent unmatched closing requirement.

3. Keywords

brackets, valid, matching, parentheses

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public boolean isValidParentheses(String s) {
    Map<Character, Character> closeToOpen = Map.of(')', '(', ']', '[', '}', '{'); // Matching pairs.
    Deque<Character> stack = new ArrayDeque<>(); // Opening brackets not yet matched.
    for (char c : s.toCharArray()) { // Scan left to right.
        if (closeToOpen.containsValue(c)) stack.push(c); // Opening bracket waits for match.
        else if (closeToOpen.containsKey(c)) { // Closing bracket must match stack top.
            if (stack.isEmpty() || stack.pop() != closeToOpen.get(c)) return false; // Mismatch.
        }
    }
    return stack.isEmpty(); // Valid only if all openings were closed.
}
```

7. Dry Run Example

Dry run the template on Valid Parentheses: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'brackets' and asks for match nested openings with the most recent unmatched closing requirement., reach for Valid Parentheses before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 20 Valid Parentheses; LC 921 Minimum Add to Make Parentheses Valid

---

Pattern 30: Monotonic Decreasing Stack

1. Pattern Name

30. Monotonic Decreasing Stack

2. Signal (when to recognize this pattern)

Find the next greater value by popping smaller values when a larger value arrives.

3. Keywords

next greater, warmer, span, stock

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public int[] nextGreater(int[] nums) {
    int[] ans = new int[nums.length]; // Result array.
    Arrays.fill(ans, -1); // Default when no greater value exists.
    Deque<Integer> stack = new ArrayDeque<>(); // Indices with decreasing values.
    for (int i = 0; i < nums.length; i++) { // Current value may resolve previous indices.
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) { // Found next greater.
            ans[stack.pop()] = nums[i]; // Assign current value to popped index.
        }
        stack.push(i); // Current index waits for a future greater value.
    }
    return ans; // Next greater values.
}
```

7. Dry Run Example

Dry run the template on Daily Temperatures: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'next greater' and asks for find the next greater value by popping smaller values when a larger value arrives., reach for Monotonic Decreasing Stack before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 739 Daily Temperatures; LC 496 Next Greater Element I

Pattern 31: Monotonic Increasing Stack

1. Pattern Name

31. Monotonic Increasing Stack

2. Signal (when to recognize this pattern)

Find the next smaller value by popping larger values when a smaller value arrives.

3. Keywords

next smaller, previous smaller, cooler

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public int[] nextSmaller(int[] nums) {
    int[] ans = new int[nums.length]; // Result array.
    Arrays.fill(ans, -1); // Default when no smaller value exists.
    Deque<Integer> stack = new ArrayDeque<>(); // Indices with increasing values.
    for (int i = 0; i < nums.length; i++) { // Current value may resolve larger previous values.
        while (!stack.isEmpty() && nums[i] < nums[stack.peek()]) { // Found next smaller.
            ans[stack.pop()] = nums[i]; // Assign current smaller value.
        }
        stack.push(i); // Current index waits for a future smaller value.
    }
    return ans; // Next smaller values.
}
```

7. Dry Run Example

Dry run the template on Next Smaller Element: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'next smaller' and asks for find the next smaller value by popping larger values when a smaller value arrives., reach for Monotonic Increasing Stack before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 84 Largest Rectangle in Histogram; LC 907 Sum of Subarray Minimums

---

Pattern 32: Histogram Stack

1. Pattern Name

32. Histogram Stack

2. Signal (when to recognize this pattern)

Use an increasing stack to know each bar's maximum rectangle boundaries.

3. Keywords

histogram, largest rectangle, area

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>(); // Indices of increasing bar heights.
    int best = 0; // Maximum area found.
    for (int i = 0; i <= heights.length; i++) { // Include sentinel position.
        int current = (i == heights.length) ? 0 : heights[i]; // Sentinel height clears stack.
        while (!stack.isEmpty() && current < heights[stack.peek()]) { // Bar at top ends here.
            int height = heights[stack.pop()]; // Height of rectangle being finalized.
            int width = stack.isEmpty() ? i : i - stack.peek() - 1; // Span between smaller bars.
            best = Math.max(best, height * width); // Update max area.
        }
        stack.push(i); // Current index may be left boundary for future bars.
    }
    return best; // Largest rectangle area.
}
```

7. Dry Run Example

Dry run the template on Largest Rectangle in Histogram: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'histogram' and asks for use an increasing stack to know each bar's maximum rectangle boundaries., reach for Histogram Stack before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 84 Largest Rectangle in Histogram; LC 85 Maximal Rectangle

---

Pattern 33: Calculator Stack

1. Pattern Name

33. Calculator Stack

2. Signal (when to recognize this pattern)

Evaluate expression state and push previous context when parentheses begin.

3. Keywords

calculator, evaluate, expression, basic calc

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public int calculateBasic(String s) {
    Deque<Integer> stack = new ArrayDeque<>(); // Stores previous result and sign.
    int result = 0, number = 0, sign = 1; // Running expression state.
    for (int i = 0; i < s.length(); i++) { // Scan every character.
        char c = s.charAt(i); // Current token.
        if (Character.isDigit(c)) number = number * 10 + (c - '0'); // Build multi-digit number.
        else if (c == '+' || c == '-') { result += sign * number; number = 0; sign = c == '+' ? 1 : -1; } //
Commit number.
        else if (c == '(') { stack.push(result); stack.push(sign); result = 0; sign = 1; } // Save context.
        else if (c == ')') { result += sign * number; number = 0; result *= stack.pop(); result += stack.pop();
} // Close context.
    }
    return result + sign * number; // Include trailing number.
}
```

7. Dry Run Example

Dry run the template on Basic Calculator: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'calculator' and asks for evaluate expression state and push previous context when parentheses begin., reach for Calculator Stack before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 224 Basic Calculator; LC 227 Basic Calculator II

Pattern 34: Decode String

1. Pattern Name

34. Decode String

2. Signal (when to recognize this pattern)

Push the current string and repeat count before entering a nested bracket.

3. Keywords

decode string, nested, repeat, encode

4. Time Complexity

O(n + output)

5. Space Complexity

O(n + output)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
public String decodeString(String s) {
    Deque<Integer> counts = new ArrayDeque<>(); // Repeat counts for nested frames.
    Deque<StringBuilder> builders = new ArrayDeque<>(); // Previous strings for frames.
    StringBuilder current = new StringBuilder(); // Current decoded segment.
    int count = 0; // Current repeat count.
    for (char c : s.toCharArray()) { // Parse one character at a time.
        if (Character.isDigit(c)) count = count * 10 + (c - '0'); // Build count.
        else if (c == '[') { counts.push(count); builders.push(current); current = new StringBuilder(); count =
0; } // Enter frame.
        else if (c == ']') { String repeated = current.toString().repeat(counts.pop()); current = builders.pop(
).append(repeated); } // Exit frame.
        else current.append(c); // Append normal character.
    }
    return current.toString(); // Fully decoded string.
}
```

7. Dry Run Example

Dry run the template on Decode String: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'decode string' and asks for push the current string and repeat count before entering a nested bracket., reach for Decode String before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 394 Decode String

Pattern 35: Stock Span

1. Pattern Name

35. Stock Span

2. Signal (when to recognize this pattern)

Use a monotonic stack of price-span pairs to aggregate dominated previous days.

3. Keywords

stock span, consecutive, online, days until

4. Time Complexity

O(1) amortized per call

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: for each element, scan outward or repeatedly rescan nested structure.

Optimized approach: keep only unresolved candidates on a stack and resolve them when the matching event appears.

The algorithm works because stack order mirrors nesting or nearest-neighbor relationships: the latest unresolved item must be handled first.

Edge cases: empty stack, unmatched brackets, duplicate values in monotonic stacks, and sentinel bars for histograms.

Java notes: use ArrayDeque instead of Stack; call push/pop/peek consistently and never push null into ArrayDeque.

```
static final class StockSpanner {
```

```
    private final Deque<int[]> stack = new ArrayDeque<>(); // Each entry is [price, span].

    int next(int price) {
        int span = 1; // Current day counts itself.
        while (!stack.isEmpty() && stack.peek()[0] <= price) { // Merge dominated previous prices.
            span += stack.pop()[1]; // Add their spans because they are consecutive.
        }
        stack.push(new int[] {price, span}); // Current price waits for a future higher price.
        return span; // Number of consecutive days <= current price.
    }
}
```

7. Dry Run Example

Dry run the template on Online Stock Span: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'stock span' and asks for use a monotonic stack of price-span pairs to aggregate dominated previous days., reach for Stock Span before designing from scratch.

9. Common Mistakes

- Using java.util.Stack instead of ArrayDeque.
- Forgetting empty-stack checks.
- Popping equal elements incorrectly in monotonic stack variants.

10. Related LeetCode Problems

LC 901 Online Stock Span

Family: Tree Patterns

Pattern 36: Tree BFS - Level Order

1. Pattern Name

36. Tree BFS - Level Order

2. Signal (when to recognize this pattern)

Process a binary tree one queue level at a time.

3. Keywords

level order, BFS, width, zigzag

4. Time Complexity

O(n)

5. Space Complexity

O(w)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>(); // Levels in traversal order.
    if (root == null) return result; // Empty tree has no levels.
    Queue<TreeNode> queue = new ArrayDeque<>(); // BFS queue.
    queue.offer(root); // Start with root.
    while (!queue.isEmpty()) { // Process one level per loop.
        int size = queue.size(); // Number of nodes in this level.
        List<Integer> level = new ArrayList<>(); // Values for current level.
        for (int i = 0; i < size; i++) { // Consume exactly this level.
            TreeNode node = queue.poll(); // Remove next node.
            level.add(node.val); // Record node value.
            if (node.left != null) queue.offer(node.left); // Queue left child.
            if (node.right != null) queue.offer(node.right); // Queue right child.
        }
        result.add(level); // Save finished level.
    }
    return result; // All levels.
}
```

7. Dry Run Example

Dry run the template on Binary Tree Level Order Traversal: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'level order' and asks for process a binary tree one queue level at a time., reach for Tree BFS - Level Order before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

Pattern 37: Tree DFS - Path Sum

1. Pattern Name

37. Tree DFS - Path Sum

2. Signal (when to recognize this pattern)

Explore root-to-leaf paths while carrying the current path and remaining sum.

3. Keywords

path sum, root to leaf, DFS, backtrack

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
    List<List<Integer>> result = new ArrayList<>(); // Valid root-to-leaf paths.
    dfsPath(root, targetSum, new ArrayList<>(), result); // Start DFS.
    return result; // All matching paths.
}

private void dfsPath(TreeNode node, int remaining, List<Integer> path, List<List<Integer>> result) {
    if (node == null) return; // Null contributes no path.
    path.add(node.val); // Choose current node.
    remaining -= node.val; // Update remaining sum.
    if (node.left == null && node.right == null && remaining == 0) result.add(new ArrayList<>(path)); // Save
leaf path.
    dfsPath(node.left, remaining, path, result); // Explore left child.
    dfsPath(node.right, remaining, path, result); // Explore right child.
    path.remove(path.size() - 1); // Undo choice for sibling paths.
}
```

7. Dry Run Example

Dry run the template on Path Sum II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'path sum' and asks for explore root-to-leaf paths while carrying the current path and remaining sum., reach for Tree DFS - Path Sum before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 112 Path Sum; LC 113 Path Sum II

Pattern 38: Tree Diameter

1. Pattern Name

38. Tree Diameter

2. Signal (when to recognize this pattern)

For every node, combine left and right subtree heights to update the best path.

3. Keywords

diameter, longest path, height, postorder

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
private int diameter; // Global best edge count.

public int diameterOfBinaryTree(TreeNode root) {
    diameter = 0; // Reset for this call.
    height(root); // Postorder computes heights and updates diameter.
    return diameter; // Longest path measured in edges.
}

private int height(TreeNode node) {
    if (node == null) return 0; // Empty subtree has height zero.
    int left = height(node.left); // Height of left subtree.
    int right = height(node.right); // Height of right subtree.
    diameter = Math.max(diameter, left + right); // Best path through this node.
    return 1 + Math.max(left, right); // Height returned to parent.
}
```

7. Dry Run Example

Dry run the template on Diameter of Binary Tree: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'diameter' and asks for for every node, combine left and right subtree heights to update the best path., reach for Tree Diameter before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 543 Diameter of Binary Tree

Pattern 39: Lowest Common Ancestor

1. Pattern Name

39. Lowest Common Ancestor

2. Signal (when to recognize this pattern)

Return targets upward; the first node receiving both sides is the ancestor.

3. Keywords

LCA, lowest common ancestor, binary tree

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root; // Base case or target found.
    TreeNode left = lowestCommonAncestor(root.left, p, q); // Search left subtree.
    TreeNode right = lowestCommonAncestor(root.right, p, q); // Search right subtree.
    if (left != null && right != null) return root; // Targets split across children.
    return left != null ? left : right; // Bubble up found target or ancestor.
}
```

7. Dry Run Example

Dry run the template on LCA of Binary Tree: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'LCA' and asks for return targets upward; the first node receiving both sides is the ancestor., reach for Lowest Common Ancestor before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 236 Lowest Common Ancestor of a Binary Tree; LC 235 LCA of BST

Pattern 40: Validate BST

1. Pattern Name

40. Validate BST

2. Signal (when to recognize this pattern)

Propagate exclusive lower and upper bounds through the tree.

3. Keywords

validate BST, inorder, sorted, bounds

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public boolean isValidBST(TreeNode root) {
    return validateBst(root, Long.MIN_VALUE, Long.MAX_VALUE); // Start with open full range.
}

private boolean validateBst(TreeNode node, long low, long high) {
    if (node == null) return true; // Empty subtree is valid.
    if (node.val <= low || node.val >= high) return false; // Violates exclusive bounds.
    return validateBst(node.left, low, node.val) && validateBst(node.right, node.val, high); // Narrow bounds.
}
```

7. Dry Run Example

Dry run the template on Validate Binary Search Tree: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'validate BST' and asks for propagate exclusive lower and upper bounds through the tree., reach for Validate BST before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 98 Validate Binary Search Tree

Pattern 41: Serialize / Deserialize Tree

1. Pattern Name

41. Serialize / Deserialize Tree

2. Signal (when to recognize this pattern)

Use preorder traversal with null markers to preserve exact tree shape.

3. Keywords

serialize, deserialize, encode tree, decode tree

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
static final class Codec {
    String serialize(TreeNode root) {
        if (root == null) return "#,"; // Null marker preserves shape.
        return root.val + "," + serialize(root.left) + serialize(root.right); // Preorder encoding.
    }

    TreeNode deserialize(String data) {
        Queue<String> tokens = new ArrayDeque<>(Arrays.asList(data.split(","))); // Token stream.
        return build(tokens); // Rebuild recursively.
    }

    private TreeNode build(Queue<String> tokens) {
        String token = tokens.poll(); // Next preorder token.
        if (token.equals("#")) return null; // Null marker becomes empty child.
        TreeNode node = new TreeNode(Integer.parseInt(token)); // Create current node.
        node.left = build(tokens); // Rebuild left subtree.
        node.right = build(tokens); // Rebuild right subtree.
        return node; // Return rebuilt subtree root.
    }
}
```

7. Dry Run Example

Dry run the template on Serialize and Deserialize Binary Tree: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'serialize' and asks for use preorder traversal with null markers to preserve exact tree shape., reach for Serialize / Deserialize Tree before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 297 Serialize and Deserialize Binary Tree

Pattern 42: Symmetric Tree

1. Pattern Name

42. Symmetric Tree

2. Signal (when to recognize this pattern)

Compare the left subtree with the right subtree in mirrored order.

3. Keywords

symmetric, mirror, same structure

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public boolean isSymmetric(TreeNode root) {
    return root == null || isMirror(root.left, root.right); // Empty tree is symmetric.
}

private boolean isMirror(TreeNode left, TreeNode right) {
    if (left == null || right == null) return left == right; // Both null is true; one null is false.
    if (left.val != right.val) return false; // Mirror values must match.
    return isMirror(left.left, right.right) && isMirror(left.right, right.left); // Cross-compare.
}
```

7. Dry Run Example

Dry run the template on Symmetric Tree: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'symmetric' and asks for compare the left subtree with the right subtree in mirrored order., reach for Symmetric Tree before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 101 Symmetric Tree

Pattern 43: Max Path Sum

1. Pattern Name

43. Max Path Sum

2. Signal (when to recognize this pattern)

At each node, keep the best single-branch gain and update global through-node sum.

3. Keywords

max path sum, any path, binary tree

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
private int maxPath; // Best path sum found.

public int maxPathSum(TreeNode root) {
    maxPath = Integer.MIN_VALUE; // Handles all-negative trees.
    maxGain(root); // Postorder computes gains.
    return maxPath; // Best any-node-to-any-node path.
}

private int maxGain(TreeNode node) {
    if (node == null) return 0; // Null contributes no gain.
    int left = Math.max(0, maxGain(node.left)); // Ignore negative left branch.
    int right = Math.max(0, maxGain(node.right)); // Ignore negative right branch.
    maxPath = Math.max(maxPath, node.val + left + right); // Path passing through node.
    return node.val + Math.max(left, right); // Single branch for parent.
}
```

7. Dry Run Example

Dry run the template on Binary Tree Maximum Path Sum: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'max path sum' and asks for at each node, keep the best single-branch gain and update global through-node sum., reach for Max Path Sum before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 124 Binary Tree Maximum Path Sum

Pattern 44: BST Insert / Delete

1. Pattern Name

44. BST Insert / Delete

2. Signal (when to recognize this pattern)

Use BST ordering to locate the node and preserve inorder ordering after changes.

3. Keywords

BST insert, BST delete, maintain BST

4. Time Complexity

O(h)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val); // Insert position found.
    if (val < root.val) root.left = insertIntoBST(root.left, val); // Insert left if smaller.
    else root.right = insertIntoBST(root.right, val); // Insert right otherwise.
    return root; // Return unchanged root pointer.
}

public TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null; // Key not found.
    if (key < root.val) root.left = deleteNode(root.left, key); // Search left.
    else if (key > root.val) root.right = deleteNode(root.right, key); // Search right.
    else if (root.left == null) return root.right; // Replace by right child.
    else if (root.right == null) return root.left; // Replace by left child.
    else { TreeNode min = root.right; while (min.left != null) min = min.left; root.val = min.val; root.right =
deleteNode(root.right, min.val); } // Use successor.
    return root; // Return updated subtree.
}
```

7. Dry Run Example

Dry run the template on Insert/Delete in BST: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'BST insert' and asks for use bst ordering to locate the node and preserve inorder ordering after changes., reach for BST Insert / Delete before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 701 Insert into a BST; LC 450 Delete Node in a BST

Pattern 45: Tree Right Side View

1. Pattern Name

45. Tree Right Side View

2. Signal (when to recognize this pattern)

During level order traversal, record the last node processed at each level.

3. Keywords

right side view, rightmost, level

4. Time Complexity

O(n)

5. Space Complexity

O(w)

6. Java 17 Template

Brute force: recompute subtree information repeatedly from each node, often O(n^2).

Optimized approach: use DFS postorder or BFS level processing so each node contributes once.

The algorithm works because tree subproblems are independent after choosing a node, and recursion returns exactly the information the parent needs.

Edge cases: null root, single-node tree, skewed trees, duplicate BST values, and all-negative path sums.

Java notes: recursion uses O(h) stack; for very skewed trees use iterative BFS/DFS if stack overflow is a concern.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> view = new ArrayList<>(); // Rightmost values per level.
    if (root == null) return view; // Empty tree.
    Queue<TreeNode> queue = new ArrayDeque<>(); // BFS queue.
    queue.offer(root); // Begin at root.
    while (!queue.isEmpty()) { // One loop per level.
        int size = queue.size(); // Nodes in level.
        for (int i = 0; i < size; i++) { // Process current level.
            TreeNode node = queue.poll(); // Next node.
            if (i == size - 1) view.add(node.val); // Last node is visible from right.
            if (node.left != null) queue.offer(node.left); // Preserve left-to-right order.
            if (node.right != null) queue.offer(node.right); // Right child enters after left.
        }
    }
    return view; // Right side view.
}
```

7. Dry Run Example

Dry run the template on Binary Tree Right Side View: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'right side view' and asks for during level order traversal, record the last node processed at each level., reach for Tree Right Side View before designing from scratch.

9. Common Mistakes

- Recomputing heights from every node.
- Using int min/max bounds for BST when node values can equal Integer.MIN_VALUE.
- Not handling null root.

10. Related LeetCode Problems

LC 199 Binary Tree Right Side View

Family: Graph Patterns

Pattern 46: Graph BFS - Shortest Path

1. Pattern Name

46. Graph BFS - Shortest Path

2. Signal (when to recognize this pattern)

In an unweighted graph or grid, BFS discovers nodes in minimum-edge order.

3. Keywords

shortest path, BFS, minimum steps, unweighted

4. Time Complexity

O(V + E)

5. Space Complexity

O(V)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public int shortestPathBinaryMatrix(int[][] grid) {
    int n = grid.length; // Square grid size.
    if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) return -1; // Blocked endpoints.
    int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}}; // Eight moves.
    Queue<int[]> queue = new ArrayDeque<>(); // BFS queue storing row, col, distance.
    queue.offer(new int[] {0, 0, 1}); // Start at top-left with path length 1.
    grid[0][0] = 1; // Mark visited in-place.
    while (!queue.isEmpty()) { // Expand by distance layers.
        int[] cur = queue.poll(); // Current cell.
        if (cur[0] == n - 1 && cur[1] == n - 1) return cur[2]; // First arrival is shortest.
        for (int[] d : dirs) { // Try all neighbors.
            int r = cur[0] + d[0], c = cur[1] + d[1]; // Neighbor coordinates.
            if (r >= 0 && c >= 0 && r < n && c < n && grid[r][c] == 0) { grid[r][c] = 1; queue.offer(new int[] {
r, c, cur[2] + 1}); } // Visit.
        }
    }
    return -1; // Destination unreachable.
}
```

7. Dry Run Example

Dry run the template on Shortest Path in Binary Matrix: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'shortest path' and asks for in an unweighted graph or grid, bfs discovers nodes in minimum-edge order., reach for Graph BFS - Shortest Path before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 1091 Shortest Path in Binary Matrix; LC 752 Open the Lock

Pattern 47: Graph DFS - Connected Components

1. Pattern Name

47. Graph DFS - Connected Components

2. Signal (when to recognize this pattern)

Start DFS/BFS from every unvisited node to count connected regions.

3. Keywords

number of islands, connected components, DFS

4. Time Complexity

O(V + E)

5. Space Complexity

O(V)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].
- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public int numIslands(char[][] grid) {
    int islands = 0; // Connected component count.
    for (int r = 0; r < grid.length; r++) { // Scan rows.
        for (int c = 0; c < grid[0].length; c++) { // Scan columns.
            if (grid[r][c] == '1') { islands++; sink(grid, r, c); } // New island found.
        }
    }
    return islands; // Total components.
}

private void sink(char[][] grid, int r, int c) {
    if (r < 0 || c < 0 || r == grid.length || c == grid[0].length || grid[r][c] != '1') return; // Stop at
water/outside.
    grid[r][c] = '0'; // Mark visited by sinking land.
    sink(grid, r + 1, c); sink(grid, r - 1, c); sink(grid, r, c + 1); sink(grid, r, c - 1); // Visit neighbors.
}
```

7. Dry Run Example

Dry run the template on Number of Islands: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'number of islands' and asks for start dfs/bfs from every unvisited node to count connected regions., reach for Graph DFS - Connected Components before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 200 Number of Islands; LC 547 Number of Provinces

Pattern 48: Topological Sort - BFS

1. Pattern Name

48. Topological Sort - BFS

2. Signal (when to recognize this pattern)

Repeatedly remove zero-indegree nodes to produce a dependency-safe order.

3. Keywords

course schedule, prerequisites, topological, indegree

4. Time Complexity

O(V + E)

5. Space Complexity

O(V + E)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>(); // Adjacency list: course -> next courses.
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>()); // Initialize lists.
    int[] indegree = new int[numCourses]; // Number of prerequisites per course.
    for (int[] edge : prerequisites) { graph.get(edge[1]).add(edge[0]); indegree[edge[0]]++; } // Build graph.
    Queue<Integer> queue = new ArrayDeque<>(); // Zero-indegree courses.
    for (int i = 0; i < numCourses; i++) if (indegree[i] == 0) queue.offer(i); // Seed queue.
    int[] order = new int[numCourses]; int idx = 0; // Output order.
    while (!queue.isEmpty()) { // Remove available courses.
        int course = queue.poll(); order[idx++] = course; // Take course.
        for (int next : graph.get(course)) if (--indegree[next] == 0) queue.offer(next); // Unlock neighbors.
    }
    return idx == numCourses ? order : new int[0]; // Empty means cycle.
}
```

7. Dry Run Example

Dry run the template on Course Schedule II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'course schedule' and asks for repeatedly remove zero-indegree nodes to produce a dependency-safe order., reach for Topological Sort - BFS before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 210 Course Schedule II; LC 207 Course Schedule

Pattern 49: Union Find (Disjoint Set)

1. Pattern Name

49. Union Find (Disjoint Set)

2. Signal (when to recognize this pattern)

Group nodes with parent pointers and answer connectivity by root identity.

3. Keywords

union find, connected components, provinces, disjoint set

4. Time Complexity

O(alpha(n)) amortized

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
static final class UnionFind {
    private final int[] parent; // parent[x] points to representative tree parent.
    private final int[] rank; // Approximate tree height.

    UnionFind(int n) {
        parent = new int[n]; rank = new int[n]; // Allocate arrays.
        for (int i = 0; i < n; i++) parent[i] = i; // Each node starts alone.
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // Path compression.
        return parent[x]; // Representative root.
    }

    boolean union(int a, int b) {
        int rootA = find(a), rootB = find(b); // Find both sets.
        if (rootA == rootB) return false; // Already connected.
        if (rank[rootA] < rank[rootB]) parent[rootA] = rootB; // Attach shorter tree.
        else if (rank[rootA] > rank[rootB]) parent[rootB] = rootA; // Attach shorter tree.
        else { parent[rootB] = rootA; rank[rootA]++; } // Tie: choose one and grow rank.
        return true; // Merge happened.
    }
}
```

7. Dry Run Example

Dry run the template on Number of Provinces: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'union find' and asks for group nodes with parent pointers and answer connectivity by root identity., reach for Union Find (Disjoint Set) before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 547 Number of Provinces; LC 684 Redundant Connection

Pattern 50: Dijkstra - Weighted Shortest Path

1. Pattern Name

50. Dijkstra - Weighted Shortest Path

2. Signal (when to recognize this pattern)

Use a min PriorityQueue to always expand the currently cheapest known node.

3. Keywords

weighted shortest, network delay, cheapest path, priority queue

4. Time Complexity

O(E log V)

5. Space Complexity

O(V + E)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public int networkDelayTime(int[][] times, int n, int source) {
    List<List<int[]>> graph = new ArrayList<>(); // 1-indexed adjacency list.
    for (int i = 0; i <= n; i++) graph.add(new ArrayList<>()); // Initialize lists.
    for (int[] e : times) graph.get(e[0]).add(new int[] {e[1], e[2]}); // edge u -> v with weight.
    int[] dist = new int[n + 1]; // Best known distance.
    Arrays.fill(dist, Integer.MAX_VALUE); // Unknown distances.
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0])); // [distance, node].
    dist[source] = 0; pq.offer(new int[] {0, source}); // Start at source.
    while (!pq.isEmpty()) { // Expand cheapest candidate.
        int[] cur = pq.poll(); int d = cur[0], u = cur[1]; // Current state.
        if (d != dist[u]) continue; // Skip stale heap entries.
        for (int[] edge : graph.get(u)) { int v = edge[0], w = edge[1]; if (d + w < dist[v]) { dist[v] = d + w;
pq.offer(new int[] {dist[v], v}); } } // Relax.
    }
    int ans = 0; for (int i = 1; i <= n; i++) ans = Math.max(ans, dist[i]); // Slowest arrival.
    return ans == Integer.MAX_VALUE ? -1 : ans; // -1 if some node unreachable.
}
```

7. Dry Run Example

Dry run the template on Network Delay Time: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'weighted shortest' and asks for use a min priorityqueue to always expand the currently cheapest known node., reach for Dijkstra - Weighted Shortest Path before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 743 Network Delay Time; LC 787 Cheapest Flights Within K Stops

Pattern 51: Bipartite Check

1. Pattern Name

51. Bipartite Check

2. Signal (when to recognize this pattern)

Two-color every connected component and reject same-color edges.

3. Keywords

bipartite, two color, graph coloring

4. Time Complexity

O(V + E)

5. Space Complexity

O(V)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public boolean isBipartite(int[][] graph) {
    int[] color = new int[graph.length]; // 0 means uncolored, 1 and -1 are two colors.
    for (int start = 0; start < graph.length; start++) { // Handle disconnected graph.
        if (color[start] != 0) continue; // Already colored component.
        Queue<Integer> queue = new ArrayDeque<>(); // BFS queue.
        queue.offer(start); color[start] = 1; // Start new component.
        while (!queue.isEmpty()) { // BFS color propagation.
            int node = queue.poll(); // Current node.
            for (int nei : graph[node]) { // Check neighbors.
                if (color[nei] == 0) { color[nei] = -color[node]; queue.offer(nei); } // Assign opposite color.
                else if (color[nei] == color[node]) return false; // Same color edge violates bipartite.
            }
        }
    }
    return true; // All components are two-colorable.
}
```

7. Dry Run Example

Dry run the template on Is Graph Bipartite?: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'bipartite' and asks for two-color every connected component and reject same-color edges., reach for Bipartite Check before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 785 Is Graph Bipartite?; LC 886 Possible Bipartition

Pattern 52: Rotting Oranges - Multi-source BFS

1. Pattern Name

52. Rotting Oranges - Multi-source BFS

2. Signal (when to recognize this pattern)

Start BFS from all sources at once so each layer is one time step.

3. Keywords

rotting oranges, multi source BFS, spread, minutes

4. Time Complexity

O(mn)

5. Space Complexity

O(mn)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length, fresh = 0; // Grid dimensions and fresh count.
    Queue<int[]> queue = new ArrayDeque<>(); // Rotten sources with time.
    for (int r = 0; r < rows; r++) for (int c = 0; c < cols; c++) { if (grid[r][c] == 2) queue.offer(new int[] {
r, c, 0}); if (grid[r][c] == 1) fresh++; } // Seed.
    int minutes = 0; int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}}; // Four directions.
    while (!queue.isEmpty()) { // Multi-source BFS.
        int[] cur = queue.poll(); minutes = Math.max(minutes, cur[2]); // Current time.
        for (int[] d : dirs) { int r = cur[0] + d[0], c = cur[1] + d[1]; if (r >= 0 && c >= 0 && r < rows && c
< cols && grid[r][c] == 1) { grid[r][c] = 2; fresh--; queue.offer(new int[] {r, c, cur[2] + 1}); } } // Rot
neighbor.
    }
    return fresh == 0 ? minutes : -1; // All fresh must rot.
}
```

7. Dry Run Example

Dry run the template on Rotting Oranges: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'rotting oranges' and asks for start bfs from all sources at once so each layer is one time step., reach for Rotting Oranges - Multi-source BFS before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 994 Rotting Oranges; LC 542 01 Matrix

Pattern 53: Cycle Detection - Directed Graph

1. Pattern Name

53. Cycle Detection - Directed Graph

2. Signal (when to recognize this pattern)

DFS colors detect back edges from a visiting node to another visiting node.

3. Keywords

cycle detection, directed, DFS colors

4. Time Complexity

O(V + E)

5. Space Complexity

O(V + E)

6. Java 17 Template

Brute force: explore all paths or rerun traversal from many starts without visited state.

Optimized approach: represent the graph with adjacency lists and use BFS/DFS/UnionFind/PriorityQueue based on edge semantics.

The algorithm works because visited state prevents repeated work, and the chosen traversal order matches the graph property being solved.

Edge cases: disconnected graphs, self-loops, cycles, empty grids, blocked starts, and one-indexed versus zero-indexed nodes.

Java notes: use List<List<Integer>> or List<List<int[]>> adjacency lists, ArrayDeque for BFS, and PriorityQueue for Dijkstra.

- Adjacency list: use List<List<Integer>> for unweighted graphs and List<List<int[]>> for weighted edges [to, weight].

```
public boolean hasDirectedCycle(int n, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>(); // Adjacency list.
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>()); // Initialize nodes.
    for (int[] e : edges) graph.get(e[0]).add(e[1]); // Add directed edge.
    int[] state = new int[n]; // 0=unvisited, 1=visiting, 2=done.
    for (int i = 0; i < n; i++) if (detectCycle(i, graph, state)) return true; // Check all components.
    return false; // No back edge found.
}

private boolean detectCycle(int node, List<List<Integer>> graph, int[] state) {
    if (state[node] == 1) return true; // Back edge to active path.
    if (state[node] == 2) return false; // Already proven safe.
    state[node] = 1; // Enter recursion stack.
    for (int next : graph.get(node)) if (detectCycle(next, graph, state)) return true; // DFS neighbors.
    state[node] = 2; // Leave recursion stack.
    return false; // No cycle below this node.
}
```

7. Dry Run Example

Dry run the template on Course Schedule: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'cycle detection' and asks for dfs colors detect back edges from a visiting node to another visiting node., reach for Cycle Detection - Directed Graph before designing from scratch.

9. Common Mistakes

- Not marking visited when enqueuing, causing duplicates.
- Building edges in the wrong prerequisite direction.
- Using DFS for weighted shortest path instead of Dijkstra.

10. Related LeetCode Problems

LC 207 Course Schedule; LC 802 Find Eventual Safe States

Family: Dynamic Programming

Pattern 54: 1D DP - Climbing Stairs

1. Pattern Name

54. 1D DP - Climbing Stairs

2. Signal (when to recognize this pattern)

The current answer depends on a fixed number of previous answers.

3. Keywords

climbing stairs, ways, steps, Fibonacci, house robber

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int climbStairs(int n) {
    if (n <= 2) return n; // Base cases: 1->1, 2->2.
    int prev2 = 1, prev1 = 2; // dp[i-2] and dp[i-1].
    for (int step = 3; step <= n; step++) { // Build from smaller answers.
        int cur = prev1 + prev2; // dp[i] = dp[i-1] + dp[i-2].
        prev2 = prev1; // Slide window.
        prev1 = cur; // Store latest answer.
    }
    return prev1; // Number of ways to reach n.
}
```

7. Dry Run Example

Dry run the template on Climbing Stairs: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'climbing stairs' and asks for the current answer depends on a fixed number of previous answers., reach for 1D DP - Climbing Stairs before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 70 Climbing Stairs; LC 198 House Robber

Pattern 55: 0/1 Knapsack

1. Pattern Name

55. 0/1 Knapsack

2. Signal (when to recognize this pattern)

Choose each item at most once to optimize value under capacity.

3. Keywords

knapsack, capacity, include exclude, 0/1

4. Time Complexity

O(nW)

5. Space Complexity

O(W)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int knapsack01(int[] weights, int[] values, int capacity) {
    int[] dp = new int[capacity + 1]; // dp[w] best value with capacity w.
    for (int i = 0; i < weights.length; i++) { // Consider each item once.
        for (int w = capacity; w >= weights[i]; w--) { // Go backward to prevent reuse.
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]); // Skip or take item.
        }
    }
    return dp[capacity]; // Best value within capacity.
}
```

7. Dry Run Example

Dry run the template on 0/1 Knapsack: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'knapsack' and asks for choose each item at most once to optimize value under capacity., reach for 0/1 Knapsack before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 416 Partition Equal Subset Sum; LC 494 Target Sum

Pattern 56: Unbounded Knapsack - Coin Change

1. Pattern Name

56. Unbounded Knapsack - Coin Change

2. Signal (when to recognize this pattern)

Items can be reused unlimited times to build a target amount.

3. Keywords

coin change, minimum coins, unbounded, amount

4. Time Complexity

O(n * amount)

5. Space Complexity

O(amount)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int coinChange(int[] coins, int amount) {
    int unreachable = amount + 1; // Sentinel larger than any possible answer.
    int[] dp = new int[amount + 1]; // dp[x] fewest coins for amount x.
    Arrays.fill(dp, unreachable); // Initially unknown.
    dp[0] = 0; // Zero coins make amount zero.
    for (int x = 1; x <= amount; x++) { // Build every amount.
        for (int coin : coins) { // Try each reusable coin.
            if (coin <= x) dp[x] = Math.min(dp[x], dp[x - coin] + 1); // Take coin last.
        }
    }
    return dp[amount] == unreachable ? -1 : dp[amount]; // Convert sentinel.
}
```

7. Dry Run Example

Dry run the template on Coin Change: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'coin change' and asks for items can be reused unlimited times to build a target amount., reach for Unbounded Knapsack - Coin Change before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 322 Coin Change; LC 518 Coin Change II

Pattern 57: Longest Common Subsequence

1. Pattern Name

57. Longest Common Subsequence

2. Signal (when to recognize this pattern)

Compare two sequences by deciding whether the current characters match.

3. Keywords

LCS, longest common, edit distance, two strings

4. Time Complexity

O(mn)

5. Space Complexity

O(mn)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int longestCommonSubsequence(String a, String b) {
    int m = a.length(), n = b.length(); // String lengths.
    int[][] dp = new int[m + 1][n + 1]; // dp[i][j] for prefixes a[0..i), b[0..j).
    for (int i = 1; i <= m; i++) { // Prefix length of a.
        for (int j = 1; j <= n; j++) { // Prefix length of b.
            if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1; // Match extends LCS.
            else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]); // Drop one char.
        }
    }
    return dp[m][n]; // LCS of full strings.
}
```

7. Dry Run Example

Dry run the template on Longest Common Subsequence: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'LCS' and asks for compare two sequences by deciding whether the current characters match., reach for Longest Common Subsequence before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 1143 Longest Common Subsequence; LC 72 Edit Distance

Pattern 58: Longest Increasing Subsequence

1. Pattern Name

58. Longest Increasing Subsequence

2. Signal (when to recognize this pattern)

Maintain the smallest possible tail for each increasing subsequence length.

3. Keywords

LIS, longest increasing, subsequence, tails

4. Time Complexity

O(n log n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int lengthOfLIS(int[] nums) {
    int[] tails = new int[nums.length]; // tails[len] is smallest tail for length len+1.
    int size = 0; // Number of active lengths.
    for (int num : nums) { // Process each value.
        int pos = Arrays.binarySearch(tails, 0, size, num); // Find replacement position.
        if (pos < 0) pos = -pos - 1; // Convert insertion point.
        tails[pos] = num; // Replace tail or append new length.
        if (pos == size) size++; // Extended LIS length.
    }
    return size; // Length of LIS.
}
```

7. Dry Run Example

Dry run the template on Longest Increasing Subsequence: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'LIS' and asks for maintain the smallest possible tail for each increasing subsequence length., reach for Longest Increasing Subsequence before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 300 Longest Increasing Subsequence; LC 354 Russian Doll Envelopes

---

Pattern 59: State Machine DP - Stock Problems

1. Pattern Name

59. State Machine DP - Stock Problems

2. Signal (when to recognize this pattern)

Model allowed actions as states and update transitions per day.

3. Keywords

stock buy sell, cooldown, states, transaction

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int maxProfitCooldown(int[] prices) {
    int hold = Integer.MIN_VALUE / 2; // Best profit while holding a stock.
    int sold = 0; // Best profit after selling today.
    int rest = 0; // Best profit while not holding and not selling today.
    for (int price : prices) { // Process each day.
        int prevHold = hold, prevSold = sold, prevRest = rest; // Freeze old states.
        hold = Math.max(prevHold, prevRest - price); // Keep holding or buy today.
        sold = prevHold + price; // Sell stock held from before.
        rest = Math.max(prevRest, prevSold); // Rest or cooldown after sale.
    }
    return Math.max(sold, rest); // Best final state cannot be holding.
}
```

7. Dry Run Example

Dry run the template on Best Time to Buy and Sell Stock with Cooldown: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'stock buy sell' and asks for model allowed actions as states and update transitions per day., reach for State Machine DP - Stock Problems before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 309 Best Time to Buy and Sell Stock with Cooldown; LC 714 With Transaction Fee

---

Pattern 60: Word Break - Prefix DP

1. Pattern Name

60. Word Break - Prefix DP

2. Signal (when to recognize this pattern)

dp[i] means the prefix ending at i can be segmented into dictionary words.

3. Keywords

word break, segment, dictionary, prefix

4. Time Complexity

O(n^2)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> words = new HashSet<>(wordDict); // O(1) dictionary lookup.
    boolean[] dp = new boolean[s.length() + 1]; // dp[i] means prefix length i is segmentable.
    dp[0] = true; // Empty prefix is valid.
    for (int end = 1; end <= s.length(); end++) { // End of current prefix.
        for (int start = 0; start < end; start++) { // Last word boundary.
            if (dp[start] && words.contains(s.substring(start, end))) { dp[end] = true; break; } // Valid split.
        }
    }
    return dp[s.length()]; // Full string segmentability.
}
```

7. Dry Run Example

Dry run the template on Word Break: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'word break' and asks for dp[i] means the prefix ending at i can be segmented into dictionary words., reach for Word Break - Prefix DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 139 Word Break; LC 140 Word Break II

Pattern 61: 2D Grid DP - Unique Paths

1. Pattern Name

61. 2D Grid DP - Unique Paths

2. Signal (when to recognize this pattern)

Grid cell answers depend on adjacent previous cells such as top and left.

3. Keywords

unique paths, grid, minimum path sum, matrix dp

4. Time Complexity

O(mn)

5. Space Complexity

O(mn)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int uniquePaths(int rows, int cols) {
    int[][] dp = new int[rows][cols]; // dp[r][c] paths to cell.
    for (int r = 0; r < rows; r++) dp[r][0] = 1; // First column has one path.
    for (int c = 0; c < cols; c++) dp[0][c] = 1; // First row has one path.
    for (int r = 1; r < rows; r++) { // Fill interior rows.
        for (int c = 1; c < cols; c++) { // Fill interior columns.
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]; // From top or left.
        }
    }
    return dp[rows - 1][cols - 1]; // Paths to destination.
}
```

7. Dry Run Example

Dry run the template on Unique Paths: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'unique paths' and asks for grid cell answers depend on adjacent previous cells such as top and left., reach for 2D Grid DP - Unique Paths before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 62 Unique Paths; LC 64 Minimum Path Sum

Pattern 62: Interval DP - Burst Balloons

1. Pattern Name

62. Interval DP - Burst Balloons

2. Signal (when to recognize this pattern)

Solve every interval by choosing the final split point inside that interval.

3. Keywords

burst balloons, interval, split, merge, strange printer

4. Time Complexity

O(n^3)

5. Space Complexity

O(n^2)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int maxCoins(int[] nums) {
    int n = nums.length + 2; // Add two virtual boundary balloons.
    int[] val = new int[n]; // Values with boundaries.
    val[0] = val[n - 1] = 1; // Boundary multiplier.
    System.arraycopy(nums, 0, val, 1, nums.length); // Copy real balloons.
    int[][] dp = new int[n][n]; // dp[left][right] best inside open interval.
    for (int len = 2; len < n; len++) { // Interval width.
        for (int left = 0; left + len < n; left++) { // Left boundary.
            int right = left + len; // Right boundary.
            for (int last = left + 1; last < right; last++) { // Last balloon burst inside.
                dp[left][right] = Math.max(dp[left][right], dp[left][last] + dp[last][right] + val[left] * val[
last] * val[right]); // Split.
            }
        }
    }
    return dp[0][n - 1]; // Best for all real balloons.
}
```

7. Dry Run Example

Dry run the template on Burst Balloons: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'burst balloons' and asks for solve every interval by choosing the final split point inside that interval., reach for Interval DP - Burst Balloons before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 312 Burst Balloons; LC 664 Strange Printer

Pattern 63: DP on Trees - House Robber III

1. Pattern Name

63. DP on Trees - House Robber III

2. Signal (when to recognize this pattern)

For each tree node, return answers for taking and skipping that node.

3. Keywords

tree DP, house robber, parent child

4. Time Complexity

O(n)

5. Space Complexity

O(h)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int robTree(TreeNode root) {
    int[] ans = robDfs(root); // ans[0]=skip root, ans[1]=take root.
    return Math.max(ans[0], ans[1]); // Best final choice.
}

private int[] robDfs(TreeNode node) {
    if (node == null) return new int[] {0, 0}; // Empty subtree has zero profit.
    int[] left = robDfs(node.left); // DP for left child.
    int[] right = robDfs(node.right); // DP for right child.
    int take = node.val + left[0] + right[0]; // Taking node forces skipping children.
    int skip = Math.max(left[0], left[1]) + Math.max(right[0], right[1]); // Skipping allows best child choices.
    return new int[] {skip, take}; // Return both states.
}
```

7. Dry Run Example

Dry run the template on House Robber III: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'tree DP' and asks for for each tree node, return answers for taking and skipping that node., reach for DP on Trees - House Robber III before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 337 House Robber III; LC 968 Binary Tree Cameras

Pattern 64: Bitmask DP

1. Pattern Name

64. Bitmask DP

2. Signal (when to recognize this pattern)

Represent a subset state with bits and transition by turning bits on.

3. Keywords

bitmask, subset, visited, TSP, shortest path all nodes

4. Time Complexity

O(2^n * n)

5. Space Complexity

O(2^n * n)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.

- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int shortestPathLength(int[][] graph) {
    int n = graph.length, full = (1 << n) - 1; // Target mask has all nodes visited.
    boolean[][] seen = new boolean[1 << n][n]; // seen[mask][node].
    Queue<int[]> queue = new ArrayDeque<>(); // [node, mask, distance].
    for (int i = 0; i < n; i++) { int mask = 1 << i; seen[mask][i] = true; queue.offer(new int[] {i, mask, 0});
} // Multi-source.
    while (!queue.isEmpty()) { // BFS over state graph.
        int[] cur = queue.poll(); // Current state.
        if (cur[1] == full) return cur[2]; // First full mask is shortest.
        for (int next : graph[cur[0]]) { int mask = cur[1] | (1 << next); if (!seen[mask][next]) { seen[mask][
next] = true; queue.offer(new int[] {next, mask, cur[2] + 1}); } } // Transition.
    }
    return 0; // Only possible when n <= 1.
}
```

7. Dry Run Example

Dry run the template on Shortest Path Visiting All Nodes: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'bitmask' and asks for represent a subset state with bits and transition by turning bits on., reach for Bitmask DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 847 Shortest Path Visiting All Nodes; LC 943 Shortest Superstring

---

Pattern 65: Memoization - Top Down DP

1. Pattern Name

65. Memoization - Top Down DP

2. Signal (when to recognize this pattern)

A recursive solution repeats states, so cache each state result once.

3. Keywords

memoization, top down, cache, recursive

4. Time Complexity

O(states * transition)

5. Space Complexity

O(states + recursion depth)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.

- Optimization: compress dimensions only when the transition no longer needs older rows or columns.
- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
private final Map<Integer, Integer> memo = new HashMap<>(); // Cache n -> answer.

public int fibMemo(int n) {
    if (n <= 1) return n; // Base cases.
    if (memo.containsKey(n)) return memo.get(n); // Cache hit.
    int value = fibMemo(n - 1) + fibMemo(n - 2); // Recurrence.
    memo.put(n, value); // Cache result before returning.
    return value; // Computed answer.
}
```

7. Dry Run Example

Dry run the template on Fibonacci / recursive DP: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'memoization' and asks for a recursive solution repeats states, so cache each state result once., reach for Memoization - Top Down DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 509 Fibonacci Number; LC 1137 N-th Tribonacci Number

Pattern 66: Palindrome DP

1. Pattern Name

66. Palindrome DP

2. Signal (when to recognize this pattern)

Use smaller substrings to determine larger palindrome substrings or subsequences.

3. Keywords

palindrome, longest palindrome, partition

4. Time Complexity

O(n^2)

5. Space Complexity

O(n^2)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int longestPalindromeSubseq(String s) {
    int n = s.length(); // String length.
    int[][] dp = new int[n][n]; // dp[i][j] best pal subsequence in s[i..j].
```

```
    for (int i = n - 1; i >= 0; i--) { // Start from shorter suffixes.
        dp[i][i] = 1; // Single character palindrome.
        for (int j = i + 1; j < n; j++) { // Expand right boundary.
            if (s.charAt(i) == s.charAt(j)) dp[i][j] = dp[i + 1][j - 1] + 2; // Pair endpoints.
            else dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]); // Drop one endpoint.
        }
    }
    return n == 0 ? 0 : dp[0][n - 1]; // Full string answer.
}
```

7. Dry Run Example

Dry run the template on Longest Palindromic Subsequence: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'palindrome' and asks for use smaller substrings to determine larger palindrome substrings or subsequences., reach for Palindrome DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 516 Longest Palindromic Subsequence; LC 647 Palindromic Substrings

---

Pattern 67: Digit DP

1. Pattern Name

67. Digit DP

2. Signal (when to recognize this pattern)

Count numbers digit by digit while tracking tightness and other constraints.

3. Keywords

digit DP, count range, digit constraint, tight

4. Time Complexity

O(digits * states)

5. Space Complexity

O(digits * states)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int countNumbersWithUniqueDigits(int n) {
    if (n == 0) return 1; // Only number 0.
    int result = 10; // All one-digit numbers.
    int uniqueForLength = 9; // Choices for first digit of current length.
    int available = 9; // Remaining digits after first digit.
    for (int len = 2; len <= n && available > 0; len++) { // Build each length.
        uniqueForLength *= available; // Choose another distinct digit.
        result += uniqueForLength; // Add numbers of this length.
```

```
        available--; // One fewer digit remains available.
    }
    return result; // Count for lengths up to n.
}
```

7. Dry Run Example

Dry run the template on Count numbers with unique digits: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'digit DP' and asks for count numbers digit by digit while tracking tightness and other constraints., reach for Digit DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 357 Count Numbers with Unique Digits; LC 902 Numbers At Most N Given Digit Set

---

Pattern 68: Partition DP

1. Pattern Name

68. Partition DP

2. Signal (when to recognize this pattern)

Optimize how to split a prefix/string into valid parts.

3. Keywords

partition, split, palindrome partition, minimum cut

4. Time Complexity

O(n^2)

5. Space Complexity

O(n^2)

6. Java 17 Template

Brute force: recursively try all choices, which repeats the same subproblems exponentially.

Optimized approach: define state, base case, transition, and evaluation order; then memoize or tabulate.

The algorithm works because each larger answer is composed from smaller already-solved states according to the recurrence.

Edge cases: zero length input, impossible states, sentinel values, overflow, and whether space can be compressed.

Java notes: initialize arrays carefully, use Integer.MAX_VALUE / 2 to avoid overflow when adding, and document state meaning.

- DP state: state variables are the smallest information needed to describe a subproblem.
- Transition: choose from previously solved states; base case seeds the table or memo.
- Optimization: compress dimensions only when the transition no longer needs older rows or columns.

```
public int minCutPalindrome(String s) {
    int n = s.length(); // String length.
    boolean[][] pal = new boolean[n][n]; // pal[i][j] true if s[i..j] palindrome.
    for (int i = n - 1; i >= 0; i--) for (int j = i; j < n; j++) pal[i][j] = s.charAt(i) == s.charAt(j) && (j -
i <= 2 || pal[i + 1][j - 1]); // Build pal table.
    int[] dp = new int[n]; // dp[i] minimum cuts for s[0..i].
    Arrays.fill(dp, Integer.MAX_VALUE); // Unknown cuts.
    for (int end = 0; end < n; end++) { // Prefix ending at end.
        if (pal[0][end]) { dp[end] = 0; continue; } // Whole prefix palindrome.
        for (int start = 1; start <= end; start++) if (pal[start][end]) dp[end] = Math.min(dp[end], dp[start -
1] + 1); // Last palindrome part.
    }
    return n == 0 ? 0 : dp[n - 1]; // Minimum cuts.
```

```
}
```

7. Dry Run Example

Dry run the template on Palindrome Partitioning II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'partition' and asks for optimize how to split a prefix/string into valid parts., reach for Partition DP before designing from scratch.

9. Common Mistakes

- Starting with a table before defining state in words.
- Using updated values from the same row when 0/1 knapsack needs backward iteration.
- Forgetting impossible-state sentinels.

10. Related LeetCode Problems

LC 132 Palindrome Partitioning II; LC 1547 Minimum Cost to Cut a Stick

---

Family: Backtracking

Pattern 69: Subsets

1. Pattern Name

69. Subsets

2. Signal (when to recognize this pattern)

Generate every include/exclude choice set.

3. Keywords

all subsets, power set, combinations

4. Time Complexity

O(2^n)

5. Space Complexity

O(n) recursion excluding output

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>(); // All subsets.
    buildSubsets(0, nums, new ArrayList<>(), result); // Start from index 0.
    return result; // Power set.
}

private void buildSubsets(int start, int[] nums, List<Integer> path, List<List<Integer>> result) {
    result.add(new ArrayList<>(path)); // Every path is a valid subset.
    for (int i = start; i < nums.length; i++) { // Choose next element.
        path.add(nums[i]); // Include nums[i].
        buildSubsets(i + 1, nums, path, result); // Recurse on remaining suffix.
        path.remove(path.size() - 1); // Backtrack include choice.
    }
}
```

7. Dry Run Example

Dry run the template on Subsets: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'all subsets' and asks for generate every include/exclude choice set., reach for Subsets before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 78 Subsets; LC 90 Subsets II

Pattern 70: Permutations

1. Pattern Name

70. Permutations

2. Signal (when to recognize this pattern)

Generate every ordering by choosing one unused value at each depth.

3. Keywords

permutations, all arrangements, ordering

4. Time Complexity

O(n!)

5. Space Complexity

O(n) recursion excluding output

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>(); // All permutations.
    boolean[] used = new boolean[nums.length]; // Tracks values already in path.
    buildPermutations(nums, used, new ArrayList<>(), result); // Start recursion.
    return result; // All orderings.
}

private void buildPermutations(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> result) {
    if (path.size() == nums.length) { result.add(new ArrayList<>(path)); return; } // Complete ordering.
    for (int i = 0; i < nums.length; i++) { // Try each unused index.
        if (used[i]) continue; // Skip already chosen value.
        used[i] = true; path.add(nums[i]); // Choose value.
        buildPermutations(nums, used, path, result); // Fill next position.
        path.remove(path.size() - 1); used[i] = false; // Undo choice.
    }
}
```

7. Dry Run Example

Dry run the template on Permutations: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'permutations' and asks for generate every ordering by choosing one unused value at each depth., reach for Permutations before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 46 Permutations; LC 47 Permutations II

Pattern 71: Combinations

1. Pattern Name

71. Combinations

2. Signal (when to recognize this pattern)

Build choices from a start index and prune when the remaining target is impossible.

3. Keywords

combination sum, choose k, target, pruning

4. Time Complexity

O(2^n)

5. Space Complexity

O(n) recursion excluding output

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    Arrays.sort(candidates); // Sorting enables pruning.
    List<List<Integer>> result = new ArrayList<>(); // Valid combinations.
    backtrackCombination(candidates, target, 0, new ArrayList<>(), result); // Start search.
    return result; // All combinations.
}

private void backtrackCombination(int[] nums, int remaining, int start, List<Integer> path, List<List<Integer>>
result) {
    if (remaining == 0) { result.add(new ArrayList<>(path)); return; } // Found exact sum.
    for (int i = start; i < nums.length && nums[i] <= remaining; i++) { // Try feasible values.
        path.add(nums[i]); // Choose candidate.
        backtrackCombination(nums, remaining - nums[i], i, path, result); // Reuse allowed.
        path.remove(path.size() - 1); // Undo choice.
    }
}
```

7. Dry Run Example

Dry run the template on Combination Sum: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'combination sum' and asks for build choices from a start index and prune when the remaining target is impossible., reach for Combinations before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 39 Combination Sum; LC 40 Combination Sum II; LC 77 Combinations

---

Pattern 72: N-Queens

1. Pattern Name

72. N-Queens

2. Signal (when to recognize this pattern)

Place one queen per row while tracking blocked columns and diagonals.

3. Keywords

N-queens, constraint, placement, diagonals

4. Time Complexity

O(n!)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>(); // Solutions.
    char[][] board = new char[n][n]; // Mutable board.
    for (char[] row : board) Arrays.fill(row, '.'); // Empty cells.
    placeQueen(0, board, new HashSet<>(), new HashSet<>(), new HashSet<>(), result); // Search.
    return result; // All valid boards.
}

private void placeQueen(int row, char[][] board, Set<Integer> cols, Set<Integer> diag1, Set<Integer> diag2,
List<List<String>> result) {
    int n = board.length; // Board size.
    if (row == n) { result.add(toBoard(board)); return; } // All queens placed.
    for (int col = 0; col < n; col++) { // Try each column.
        if (cols.contains(col) || diag1.contains(row - col) || diag2.contains(row + col)) continue; // Attacked.
        cols.add(col); diag1.add(row - col); diag2.add(row + col); board[row][col] = 'Q'; // Choose.
        placeQueen(row + 1, board, cols, diag1, diag2, result); // Next row.
        board[row][col] = '.'; cols.remove(col); diag1.remove(row - col); diag2.remove(row + col); // Undo.
    }
}

private List<String> toBoard(char[][] board) {
    List<String> rows = new ArrayList<>(); // Encoded board.
    for (char[] row : board) rows.add(new String(row)); // Convert each row.
    return rows; // Board representation.
}
```

7. Dry Run Example

Dry run the template on N-Queens: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'N-queens' and asks for place one queen per row while tracking blocked columns and diagonals., reach for N-Queens before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 51 N-Queens; LC 52 N-Queens II

Pattern 73: Sudoku Solver

1. Pattern Name

73. Sudoku Solver

2. Signal (when to recognize this pattern)

Fill the next empty cell with a valid digit and backtrack on contradiction.

3. Keywords

sudoku, fill grid, constraint, rows columns boxes

4. Time Complexity

O(9^m)

5. Space Complexity

O(1) extra aside from recursion

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public void solveSudoku(char[][] board) {
    solveCell(board); // Mutates board in place.
}

private boolean solveCell(char[][] board) {
    for (int r = 0; r < 9; r++) { // Find first empty row.
        for (int c = 0; c < 9; c++) { // Find first empty column.
            if (board[r][c] != '.') continue; // Skip filled cells.
            for (char val = '1'; val <= '9'; val++) { // Try every digit.
                if (isValidSudokuMove(board, r, c, val)) { board[r][c] = val; if (solveCell(board)) return true;
 board[r][c] = '.'; } // Choose/recurse/undo.
            }
            return false; // No digit works here.
        }
    }
    return true; // No empty cells remain.
}

private boolean isValidSudokuMove(char[][] board, int row, int col, char val) {
    for (int i = 0; i < 9; i++) { // Check row, column, and box.
        if (board[row][i] == val || board[i][col] == val) return false; // Row or column conflict.
        int r = 3 * (row / 3) + i / 3, c = 3 * (col / 3) + i % 3; // Box cell.
        if (board[r][c] == val) return false; // Box conflict.
    }
    return true; // Placement is valid.
}
```

7. Dry Run Example

Dry run the template on Sudoku Solver: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'sudoku' and asks for fill the next empty cell with a valid digit and backtrack on contradiction., reach for Sudoku Solver before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 37 Sudoku Solver; LC 36 Valid Sudoku

Pattern 74: Word Search

1. Pattern Name

74. Word Search

2. Signal (when to recognize this pattern)

DFS through adjacent cells while marking the current path as visited.

3. Keywords

word search, path, grid, adjacent

4. Time Complexity

O(mn * 4^L)

5. Space Complexity

O(L)

6. Java 17 Template

Brute force: generate every raw possibility and validate only at the end.

Optimized approach: build partial candidates, reject invalid branches early, and undo each choice after recursion.

The algorithm works because the recursion tree enumerates each legal decision path exactly once while pruning impossible prefixes.

Edge cases: duplicate candidates, empty result, mutation restoration, and constraints that fail at the first choice.

Java notes: copy path with new ArrayList<>(path) when saving, and restore arrays/sets before returning from recursion.

- Recursion tree: each call handles one node, cell, or choice; stack space equals the maximum recursion depth.

```
public boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++) { // Try every start row.
        for (int c = 0; c < board[0].length; c++) { // Try every start column.
            if (searchWord(board, word, r, c, 0)) return true; // Found path.
        }
    }
    return false; // No path spells word.
}

private boolean searchWord(char[][] board, String word, int r, int c, int idx) {
    if (idx == word.length()) return true; // All characters matched.
    if (r < 0 || c < 0 || r == board.length || c == board[0].length || board[r][c] != word.charAt(idx)) return
false; // Invalid.
    char saved = board[r][c]; // Save character before marking.
    board[r][c] = '#'; // Mark visited in current path.
    boolean found = searchWord(board, word, r + 1, c, idx + 1) || searchWord(board, word, r - 1, c, idx + 1) ||
searchWord(board, word, r, c + 1, idx + 1) || searchWord(board, word, r, c - 1, idx + 1); // Explore.
    board[r][c] = saved; // Restore for other paths.
    return found; // Whether any direction worked.
}
```

7. Dry Run Example

Dry run the template on Word Search: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'word search' and asks for dfs through adjacent cells while marking the current path as visited., reach for Word Search before designing from scratch.

9. Common Mistakes

- Saving the mutable path reference instead of a copy.
- Forgetting to undo a choice.
- Not pruning after sorting candidates.

10. Related LeetCode Problems

LC 79 Word Search; LC 212 Word Search II

Family: Heap / Priority Queue

Pattern 75: Top K Elements

1. Pattern Name

75. Top K Elements

2. Signal (when to recognize this pattern)

Keep only k best candidates in a heap, or use frequency plus heap ordering.

3. Keywords

top k, kth largest, k frequent, heap

4. Time Complexity

O(n log k)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: sort the full collection after every update or scan all candidates each time.

Optimized approach: use PriorityQueue to keep only the currently best candidates or next sorted heads.

The algorithm works because the heap root always exposes the next item needed under the comparator.

Edge cases: k equals zero, k greater than unique count, stale heap entries, and comparator overflow.

Java notes: use PriorityQueue with Comparator.comparingInt; for max heap use Comparator.reverseOrder or reversed comparator.

- Heap implementation: Java PriorityQueue is a min heap by default; provide a comparator for max heap or custom ordering.

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>(); // Number -> frequency.
    for (int num : nums) freq.merge(num, 1, Integer::sum); // Count values.
    PriorityQueue<int[]> heap = new PriorityQueue<>(Comparator.comparingInt(a -> a[1])); // Min heap by
frequency.
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) { // Process unique values.
        heap.offer(new int[] {e.getKey(), e.getValue()}); // Add candidate.
        if (heap.size() > k) heap.poll(); // Keep only top k frequencies.
    }
    int[] ans = new int[heap.size()]; // Result values.
    for (int i = ans.length - 1; i >= 0; i--) ans[i] = heap.poll()[0]; // Extract.
    return ans; // Top k frequent values.
}
```

7. Dry Run Example

Dry run the template on Top K Frequent Elements: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'top k' and asks for keep only k best candidates in a heap, or use frequency plus heap ordering., reach for Top K Elements before designing from scratch.

9. Common Mistakes

- Comparator overflow from a[0] - b[0].
- Letting heap size grow beyond k.
- Forgetting stale entry checks in Dijkstra.

10. Related LeetCode Problems

LC 347 Top K Frequent Elements; LC 215 Kth Largest Element in an Array

---

Pattern 76: K-way Merge

1. Pattern Name

76. K-way Merge

2. Signal (when to recognize this pattern)

Use a min heap containing the current head of each sorted input.

3. Keywords

merge k lists, k way merge, sorted, priority queue

4. Time Complexity

O(n log k)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: sort the full collection after every update or scan all candidates each time.

Optimized approach: use PriorityQueue to keep only the currently best candidates or next sorted heads.

The algorithm works because the heap root always exposes the next item needed under the comparator.

Edge cases: k equals zero, k greater than unique count, stale heap entries, and comparator overflow.

Java notes: use PriorityQueue with Comparator.comparingInt; for max heap use Comparator.reverseOrder or reversed comparator.

- Heap implementation: Java PriorityQueue is a min heap by default; provide a comparator for max heap or custom ordering.

```
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>(Comparator.comparingInt(node -> node.val)); // Min heap
by node value.
    for (ListNode node : lists) if (node != null) heap.offer(node); // Add each list head.
    ListNode dummy = new ListNode(0); // Dummy simplifies appending.
    ListNode tail = dummy; // Tail of merged list.
    while (!heap.isEmpty()) { // Repeatedly take smallest head.
        ListNode node = heap.poll(); // Current smallest node.
        tail.next = node; tail = tail.next; // Append to output.
        if (node.next != null) heap.offer(node.next); // Add next from same list.
    }
    return dummy.next; // Merged sorted list.
}
```

7. Dry Run Example

Dry run the template on Merge K Sorted Lists: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'merge k lists' and asks for use a min heap containing the current head of each sorted input., reach for K-way Merge before designing from scratch.

9. Common Mistakes

- Comparator overflow from a[0] - b[0].
- Letting heap size grow beyond k.
- Forgetting stale entry checks in Dijkstra.

10. Related LeetCode Problems

LC 23 Merge K Sorted Lists; LC 373 Find K Pairs with Smallest Sums

---

Pattern 77: Two Heaps - Median

1. Pattern Name

77. Two Heaps - Median

2. Signal (when to recognize this pattern)

Maintain lower half in a max heap and upper half in a min heap.

3. Keywords

median, data stream, two heaps, online

4. Time Complexity

O(log n) add, O(1) find

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: sort the full collection after every update or scan all candidates each time.

Optimized approach: use PriorityQueue to keep only the currently best candidates or next sorted heads.

The algorithm works because the heap root always exposes the next item needed under the comparator.

Edge cases: k equals zero, k greater than unique count, stale heap entries, and comparator overflow.

Java notes: use PriorityQueue with Comparator.comparingInt; for max heap use Comparator.reverseOrder or reversed comparator.

- Heap implementation: Java PriorityQueue is a min heap by default; provide a comparator for max heap or custom ordering.

```
static final class MedianFinder {
    private final PriorityQueue<Integer> lower = new PriorityQueue<>(Comparator.reverseOrder()); // Max heap.
    private final PriorityQueue<Integer> upper = new PriorityQueue<>(); // Min heap.

    void addNum(int num) {
        lower.offer(num); // Add to lower half first.
        upper.offer(lower.poll()); // Move largest lower value to upper half.
        if (upper.size() > lower.size()) lower.offer(upper.poll()); // Rebalance sizes.
    }

    double findMedian() {
        if (lower.size() > upper.size()) return lower.peek(); // Odd count median.
        return (lower.peek() + upper.peek()) / 2.0; // Even count average.
    }
}
```

7. Dry Run Example

Dry run the template on Find Median from Data Stream: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'median' and asks for maintain lower half in a max heap and upper half in a min heap., reach for Two Heaps - Median before designing from scratch.

9. Common Mistakes

- Comparator overflow from a[0] - b[0].
- Letting heap size grow beyond k.
- Forgetting stale entry checks in Dijkstra.

10. Related LeetCode Problems

LC 295 Find Median from Data Stream; LC 480 Sliding Window Median

---

Pattern 78: Task Scheduler

1. Pattern Name

78. Task Scheduler

2. Signal (when to recognize this pattern)

Use the highest task frequency to compute idle slots or simulate with a heap.

3. Keywords

task scheduler, cooldown, CPU, frequency

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: sort the full collection after every update or scan all candidates each time.

Optimized approach: use PriorityQueue to keep only the currently best candidates or next sorted heads.

The algorithm works because the heap root always exposes the next item needed under the comparator.

Edge cases: k equals zero, k greater than unique count, stale heap entries, and comparator overflow.

Java notes: use PriorityQueue with Comparator.comparingInt; for max heap use Comparator.reverseOrder or reversed comparator.

- Heap implementation: Java PriorityQueue is a min heap by default; provide a comparator for max heap or custom ordering.

```
public int leastInterval(char[] tasks, int cooldown) {
    int[] freq = new int[26]; // Task frequencies.
    for (char task : tasks) freq[task - 'A']++; // Count tasks.
    int maxFreq = 0, maxCount = 0; // Highest frequency and how many tasks have it.
    for (int f : freq) { // Inspect all task types.
        if (f > maxFreq) { maxFreq = f; maxCount = 1; } // New maximum.
        else if (f == maxFreq) maxCount++; // Tie for maximum.
    }
    int slots = (maxFreq - 1) * (cooldown + 1) + maxCount; // Minimum schedule frame.
    return Math.max(tasks.length, slots); // Other tasks can fill idle slots.
}
```

7. Dry Run Example

Dry run the template on Task Scheduler: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'task scheduler' and asks for use the highest task frequency to compute idle slots or simulate with a heap., reach for Task Scheduler before designing from scratch.

9. Common Mistakes

- Comparator overflow from a[0] - b[0].
- Letting heap size grow beyond k.
- Forgetting stale entry checks in Dijkstra.

10. Related LeetCode Problems

LC 621 Task Scheduler

Pattern 79: Sliding Window Maximum

1. Pattern Name

79. Sliding Window Maximum

2. Signal (when to recognize this pattern)

Use a decreasing deque so the front is always the current window maximum.

3. Keywords

sliding window max, deque, maximum, monotonic queue

4. Time Complexity

O(n)

5. Space Complexity

O(k)

6. Java 17 Template

Brute force: sort the full collection after every update or scan all candidates each time.

Optimized approach: use PriorityQueue to keep only the currently best candidates or next sorted heads.

The algorithm works because the heap root always exposes the next item needed under the comparator.

Edge cases: k equals zero, k greater than unique count, stale heap entries, and comparator overflow.

Java notes: use PriorityQueue with Comparator.comparingInt; for max heap use Comparator.reverseOrder or reversed comparator.

- Heap implementation: Java PriorityQueue is a min heap by default; provide a comparator for max heap or custom ordering.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    if (nums.length == 0 || k == 0) return new int[0]; // Edge guard.
    int[] ans = new int[nums.length - k + 1]; // One answer per full window.
    Deque<Integer> deque = new ArrayDeque<>(); // Indices with decreasing values.
    for (int i = 0; i < nums.length; i++) { // Slide right edge.
        while (!deque.isEmpty() && deque.peekFirst() <= i - k) deque.pollFirst(); // Remove stale indices.
        while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i]) deque.pollLast(); // Remove smaller
values.
        deque.offerLast(i); // Add current index.
        if (i >= k - 1) ans[i - k + 1] = nums[deque.peekFirst()]; // Front is max.
    }
    return ans; // Window maxima.
}
```

7. Dry Run Example

Dry run the template on Sliding Window Maximum: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'sliding window max' and asks for use a decreasing deque so the front is always the current window maximum., reach for Sliding Window Maximum before designing from scratch.

9. Common Mistakes

- Comparator overflow from a[0] - b[0].
- Letting heap size grow beyond k.
- Forgetting stale entry checks in Dijkstra.

10. Related LeetCode Problems

LC 239 Sliding Window Maximum

Family: Intervals

Pattern 80: Merge Intervals

1. Pattern Name

80. Merge Intervals

2. Signal (when to recognize this pattern)

Sort by start time and merge into the last output interval when overlapping.

3. Keywords

merge intervals, overlapping, combine

4. Time Complexity

O(n log n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: compare every interval with every other interval.

Optimized approach: sort by start or end time, then sweep once while maintaining the active or last-kept interval.

The algorithm works because sorted order makes overlap decisions local: only the current active end matters.

Edge cases: empty interval list, touching endpoints, inclusive versus exclusive ends, and mutation of input intervals.

Java notes: sort int[][] with Comparator.comparingInt(a -> a[0]) or a[1], and clone intervals if callers must keep inputs unchanged.

```
public int[][] mergeIntervals(int[][] intervals) {
    if (intervals.length <= 1) return intervals; // Already merged.
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[0])); // Sort by start.
    List<int[]> merged = new ArrayList<>(); // Output intervals.
    for (int[] cur : intervals) { // Process intervals in start order.
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < cur[0]) merged.add(cur.clone()); // No
overlap.
        else merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() - 1)[1], cur[1]); // Merge
overlap.
    }
    return merged.toArray(new int[merged.size()][]); // Convert to array.
}
```

7. Dry Run Example

Dry run the template on Merge Intervals: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'merge intervals' and asks for sort by start time and merge into the last output interval when overlapping., reach for Merge Intervals before designing from scratch.

9. Common Mistakes

- Sorting by the wrong endpoint for the greedy goal.
- Treating touching intervals incorrectly.
- Mutating input intervals unexpectedly.

10. Related LeetCode Problems

LC 56 Merge Intervals

Pattern 81: Insert Interval

1. Pattern Name

81. Insert Interval

2. Signal (when to recognize this pattern)

Add intervals before, merge overlaps with the new interval, then append the rest.

3. Keywords

insert interval, add interval, sorted intervals

4. Time Complexity

O(n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: compare every interval with every other interval.

Optimized approach: sort by start or end time, then sweep once while maintaining the active or last-kept interval.

The algorithm works because sorted order makes overlap decisions local: only the current active end matters.

Edge cases: empty interval list, touching endpoints, inclusive versus exclusive ends, and mutation of input intervals.

Java notes: sort int[][] with Comparator.comparingInt(a -> a[0]) or a[1], and clone intervals if callers must keep inputs unchanged.

```
public int[][] insertInterval(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>(); // Output list.
    int i = 0; // Current interval index.
    while (i < intervals.length && intervals[i][1] < newInterval[0]) result.add(intervals[i++]); // Add
intervals before.
    while (i < intervals.length && intervals[i][0] <= newInterval[1]) { // Merge overlaps.
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]); // Expand start.
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]); // Expand end.
        i++; // Consume merged interval.
    }
    result.add(newInterval); // Add merged new interval.
    while (i < intervals.length) result.add(intervals[i++]); // Add intervals after.
    return result.toArray(new int[result.size()][]); // Convert to array.
}
```

7. Dry Run Example

Dry run the template on Insert Interval: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'insert interval' and asks for add intervals before, merge overlaps with the new interval, then append the rest., reach for Insert Interval before designing from scratch.

9. Common Mistakes

- Sorting by the wrong endpoint for the greedy goal.
- Treating touching intervals incorrectly.
- Mutating input intervals unexpectedly.

10. Related LeetCode Problems

LC 57 Insert Interval

Pattern 82: Meeting Rooms II

1. Pattern Name

82. Meeting Rooms II

2. Signal (when to recognize this pattern)

Track meeting start and end events to count concurrent meetings.

3. Keywords

meeting rooms, minimum rooms, schedule, sweep line

4. Time Complexity

O(n log n)

5. Space Complexity

O(n)

6. Java 17 Template

Brute force: compare every interval with every other interval.

Optimized approach: sort by start or end time, then sweep once while maintaining the active or last-kept interval.

The algorithm works because sorted order makes overlap decisions local: only the current active end matters.

Edge cases: empty interval list, touching endpoints, inclusive versus exclusive ends, and mutation of input intervals.

Java notes: sort int[][] with Comparator.comparingInt(a -> a[0]) or a[1], and clone intervals if callers must keep inputs unchanged.

```
public int minMeetingRooms(int[][] intervals) {
    int n = intervals.length; // Number of meetings.
    int[] starts = new int[n], ends = new int[n]; // Separate start/end events.
    for (int i = 0; i < n; i++) { starts[i] = intervals[i][0]; ends[i] = intervals[i][1]; } // Fill arrays.
    Arrays.sort(starts); Arrays.sort(ends); // Chronological event order.
    int rooms = 0, best = 0, end = 0; // Active rooms and end pointer.
    for (int start : starts) { // Process meetings by start time.
        while (end < n && ends[end] <= start) { rooms--; end++; } // Free ended rooms.
        rooms++; // Allocate room for current meeting.
        best = Math.max(best, rooms); // Track peak concurrency.
    }
    return best; // Minimum rooms required.
}
```

7. Dry Run Example

Dry run the template on Meeting Rooms II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'meeting rooms' and asks for track meeting start and end events to count concurrent meetings., reach for Meeting Rooms II before designing from scratch.

9. Common Mistakes

- Sorting by the wrong endpoint for the greedy goal.
- Treating touching intervals incorrectly.
- Mutating input intervals unexpectedly.

10. Related LeetCode Problems

LC 253 Meeting Rooms II; LC 252 Meeting Rooms

Pattern 83: Non-overlapping Intervals

1. Pattern Name

83. Non-overlapping Intervals

2. Signal (when to recognize this pattern)

Greedily keep intervals with earliest end time to minimize removals.

3. Keywords

non-overlapping, minimum remove, erase, greedy intervals

4. Time Complexity

O(n log n)

5. Space Complexity

O(1) extra

6. Java 17 Template

Brute force: compare every interval with every other interval.

Optimized approach: sort by start or end time, then sweep once while maintaining the active or last-kept interval.

The algorithm works because sorted order makes overlap decisions local: only the current active end matters.

Edge cases: empty interval list, touching endpoints, inclusive versus exclusive ends, and mutation of input intervals.

Java notes: sort int[][] with Comparator.comparingInt(a -> a[0]) or a[1], and clone intervals if callers must keep inputs unchanged.

```
public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0; // No removals.
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[1])); // Keep earliest ending intervals.
    int removed = 0; // Count intervals removed.
    int prevEnd = intervals[0][1]; // End of last kept interval.
    for (int i = 1; i < intervals.length; i++) { // Scan remaining intervals.
        if (intervals[i][0] < prevEnd) removed++; // Overlap: remove current interval.
        else prevEnd = intervals[i][1]; // Non-overlap: keep current interval.
    }
    return removed; // Minimum removals.
}
```

7. Dry Run Example

Dry run the template on Non-overlapping Intervals: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'non-overlapping' and asks for greedily keep intervals with earliest end time to minimize removals., reach for Non-overlapping Intervals before designing from scratch.

9. Common Mistakes

- Sorting by the wrong endpoint for the greedy goal.
- Treating touching intervals incorrectly.
- Mutating input intervals unexpectedly.

10. Related LeetCode Problems

LC 435 Non-overlapping Intervals; LC 452 Minimum Number of Arrows

---

Pattern 84: Minimum Platforms

1. Pattern Name

84. Minimum Platforms

2. Signal (when to recognize this pattern)

Sweep sorted arrivals and departures to track maximum simultaneous trains.

3. Keywords

minimum platforms, trains, station, arrivals departures

4. Time Complexity

O(n log n)

5. Space Complexity

O(1) extra after sorting

6. Java 17 Template

Brute force: compare every interval with every other interval.

Optimized approach: sort by start or end time, then sweep once while maintaining the active or last-kept interval.

The algorithm works because sorted order makes overlap decisions local: only the current active end matters.

Edge cases: empty interval list, touching endpoints, inclusive versus exclusive ends, and mutation of input intervals.

Java notes: sort int[][] with Comparator.comparingInt(a -> a[0]) or a[1], and clone intervals if callers must keep inputs unchanged.

```
public int minPlatforms(int[] arrivals, int[] departures) {
    Arrays.sort(arrivals); Arrays.sort(departures); // Sort event times.
    int platforms = 0, best = 0, i = 0, j = 0; // Active trains and pointers.
    while (i < arrivals.length && j < departures.length) { // Sweep both arrays.
        if (arrivals[i] <= departures[j]) { platforms++; i++; best = Math.max(best, platforms); } // Train
arrives before platform frees.
        else { platforms--; j++; } // A train departed, freeing a platform.
    }
    return best; // Peak platforms needed.
}
```

7. Dry Run Example

Dry run the template on Minimum Number of Platforms: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'minimum platforms' and asks for sweep sorted arrivals and departures to track maximum simultaneous trains., reach for Minimum Platforms before designing from scratch.

9. Common Mistakes

- Sorting by the wrong endpoint for the greedy goal.
- Treating touching intervals incorrectly.
- Mutating input intervals unexpectedly.

10. Related LeetCode Problems

GFG Minimum Platforms; LC 253 Meeting Rooms II

Family: Greedy

Pattern 85: Jump Game

1. Pattern Name

85. Jump Game

2. Signal (when to recognize this pattern)

Maintain the farthest reachable index, or expand jump levels greedily.

3. Keywords

jump game, reach, reachable, greedy, farthest

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try every sequence of choices or starting point.

Optimized approach: prove a local choice preserves the possibility of a global optimum.

The algorithm works because each greedy choice discards only candidates that are dominated by the chosen state.

Edge cases: unreachable states, zero jumps, negative net gas, and empty activity lists.

Java notes: keep scalar invariants explicit and avoid unnecessary collections.

```
public boolean canJump(int[] nums) {
    int farthest = 0; // Farthest reachable index so far.
    for (int i = 0; i < nums.length; i++) { // Scan positions in order.
        if (i > farthest) return false; // Cannot reach this index.
        farthest = Math.max(farthest, i + nums[i]); // Extend reach.
    }
    return true; // Every index up to end was reachable.
}

public int minJumps(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0; // BFS-like layer state.
    for (int i = 0; i < nums.length - 1; i++) { // No need to jump from last index.
        farthest = Math.max(farthest, i + nums[i]); // Best next layer reach.
        if (i == currentEnd) { jumps++; currentEnd = farthest; } // Finish current jump layer.
    }
    return jumps; // Minimum jumps.
}
```

7. Dry Run Example

Dry run the template on Jump Game I and II: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'jump game' and asks for maintain the farthest reachable index, or expand jump levels greedily., reach for Jump Game before designing from scratch.

9. Common Mistakes

- Applying greedy without an exchange argument.
- Resetting state too late.
- Ignoring global feasibility checks.

10. Related LeetCode Problems

LC 55 Jump Game; LC 45 Jump Game II

Pattern 86: Gas Station

1. Pattern Name

86. Gas Station

2. Signal (when to recognize this pattern)

If a start fails at i, every station before i also fails, so restart after i.

3. Keywords

gas station, circular, complete route, total tank

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: try every sequence of choices or starting point.

Optimized approach: prove a local choice preserves the possibility of a global optimum.

The algorithm works because each greedy choice discards only candidates that are dominated by the chosen state.

Edge cases: unreachable states, zero jumps, negative net gas, and empty activity lists.

Java notes: keep scalar invariants explicit and avoid unnecessary collections.

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int total = 0; // Net gas over entire route.
    int tank = 0; // Net gas from current start.
    int start = 0; // Candidate starting station.
    for (int i = 0; i < gas.length; i++) { // Visit stations in order.
        int diff = gas[i] - cost[i]; // Net gain at station i.
        total += diff; // Track global feasibility.
        tank += diff; // Track current route feasibility.
        if (tank < 0) { start = i + 1; tank = 0; } // Current start failed; restart after i.
    }
    return total >= 0 ? start : -1; // Need nonnegative total to complete circle.
}
```

7. Dry Run Example

Dry run the template on Gas Station: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'gas station' and asks for if a start fails at i, every station before i also fails, so restart after i., reach for Gas Station before designing from scratch.

9. Common Mistakes

- Applying greedy without an exchange argument.
- Resetting state too late.
- Ignoring global feasibility checks.

10. Related LeetCode Problems

LC 134 Gas Station

Pattern 87: Greedy Interval Scheduling

1. Pattern Name

87. Greedy Interval Scheduling

2. Signal (when to recognize this pattern)

Choose the activity that ends earliest to leave maximum room for the rest.

3. Keywords

activity selection, maximum activities, end time

4. Time Complexity

O(n log n)

5. Space Complexity

O(1) extra

6. Java 17 Template

Brute force: try every sequence of choices or starting point.

Optimized approach: prove a local choice preserves the possibility of a global optimum.

The algorithm works because each greedy choice discards only candidates that are dominated by the chosen state.

Edge cases: unreachable states, zero jumps, negative net gas, and empty activity lists.

Java notes: keep scalar invariants explicit and avoid unnecessary collections.

```
public int maxNonOverlappingActivities(int[][] activities) {
    if (activities.length == 0) return 0; // No activities.
    Arrays.sort(activities, Comparator.comparingInt(a -> a[1])); // Earliest finish first.
    int count = 1; // Keep first activity.
    int lastEnd = activities[0][1]; // End time of last kept activity.
    for (int i = 1; i < activities.length; i++) { // Consider remaining activities.
        if (activities[i][0] >= lastEnd) { count++; lastEnd = activities[i][1]; } // Compatible, keep it.
    }
    return count; // Maximum compatible activities.
}
```

7. Dry Run Example

Dry run the template on Maximum non-overlapping activities: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'activity selection' and asks for choose the activity that ends earliest to leave maximum room for the rest., reach for Greedy Interval Scheduling before designing from scratch.

9. Common Mistakes

- Applying greedy without an exchange argument.
- Resetting state too late.
- Ignoring global feasibility checks.

10. Related LeetCode Problems

LC 1353 Maximum Number of Events That Can Be Attended; LC 435 Non-overlapping Intervals

---

Family: Trie

Pattern 88: Basic Trie

1. Pattern Name

88. Basic Trie

2. Signal (when to recognize this pattern)

Many operations share prefixes, so store characters as paths from a root.

3. Keywords

trie, prefix, autocomplete, search, startsWith

4. Time Complexity

O(L) per operation

5. Space Complexity

O(total characters)

6. Java 17 Template

Brute force: compare every word or pair character-by-character for each query.

Optimized approach: store shared prefixes or bits in a tree so each query follows only one path per character or bit.

The algorithm works because each edge represents one prefix decision, and all strings/numbers sharing that prefix reuse the same nodes.

Edge cases: empty string, missing prefix edge, duplicate insertions, signed integers, and memory usage.

Java notes: use HashMap<Character, TrieNode> for general alphabets, arrays for fixed alphabets, and unsigned shift >>> for bits.

- Trie implementation: TrieNode stores children and an end marker; Trie owns the root and exposes insert/search/prefix methods.

```
static final class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>(); // Outgoing character edges.
    boolean isWord; // True when a complete word ends here.
}

static final class Trie {
    private final TrieNode root = new TrieNode(); // Empty root node.

    void insert(String word) {
        TrieNode node = root; // Start at root.
        for (char c : word.toCharArray()) node = node.children.computeIfAbsent(c, ignored -> new TrieNode()); //
 Create/follow edge.
        node.isWord = true; // Mark complete word.
    }

    boolean search(String word) {
        TrieNode node = findNode(word); // Follow all characters.
        return node != null && node.isWord; // Must end at a word.
    }

    boolean startsWith(String prefix) {
        return findNode(prefix) != null; // Prefix exists if path exists.
    }

    private TrieNode findNode(String s) {
        TrieNode node = root; // Start at root.
        for (char c : s.toCharArray()) { node = node.children.get(c); if (node == null) return null; } //
Follow edges.
        return node; // Final node.
    }
}
```

7. Dry Run Example

Dry run the template on Implement Trie: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'trie' and asks for many operations share prefixes, so store characters as paths from a root., reach for Basic Trie before designing from scratch.

9. Common Mistakes

- Not marking word endings.
- Using char arithmetic that only works for lowercase when input is broader.
- Forgetting signed bit behavior.

10. Related LeetCode Problems

LC 208 Implement Trie; LC 1268 Search Suggestions System

Pattern 89: XOR Trie

1. Pattern Name

89. XOR Trie

2. Signal (when to recognize this pattern)

Walk opposite bits first to maximize XOR at each bit position.

3. Keywords

XOR trie, maximum XOR, bits, binary trie

4. Time Complexity

O(n * B)

5. Space Complexity

O(n * B)

6. Java 17 Template

Brute force: compare every word or pair character-by-character for each query.

Optimized approach: store shared prefixes or bits in a tree so each query follows only one path per character or bit.

The algorithm works because each edge represents one prefix decision, and all strings/numbers sharing that prefix reuse the same nodes.

Edge cases: empty string, missing prefix edge, duplicate insertions, signed integers, and memory usage.

Java notes: use HashMap<Character, TrieNode> for general alphabets, arrays for fixed alphabets, and unsigned shift >>> for bits.

- Trie implementation: TrieNode stores children and an end marker; Trie owns the root and exposes insert/search/prefix methods.

```
static final class BitTrieNode {
    BitTrieNode[] child = new BitTrieNode[2]; // child[0] and child[1].
}

public int findMaximumXOR(int[] nums) {
    BitTrieNode root = new BitTrieNode(); // Root of binary trie.
    for (int num : nums) insertBits(root, num); // Insert every number.
    int best = 0; // Maximum XOR found.
    for (int num : nums) best = Math.max(best, queryMaxXor(root, num)); // Query best partner.
    return best; // Maximum XOR pair value.
}

private void insertBits(BitTrieNode root, int num) {
    BitTrieNode node = root; // Start at root.
    for (int bit = 31; bit >= 0; bit--) { int b = (num >>> bit) & 1; if (node.child[b] == null) node.child[b] =
new BitTrieNode(); node = node.child[b]; } // Insert bits.
}

private int queryMaxXor(BitTrieNode root, int num) {
    BitTrieNode node = root; int xor = 0; // Query state.
    for (int bit = 31; bit >= 0; bit--) { int b = (num >>> bit) & 1, want = 1 - b; if (node.child[want] !=
null) { xor |= 1 << bit; node = node.child[want]; } else node = node.child[b]; } // Prefer opposite bit.
    return xor; // Best XOR against inserted numbers.
}
```

7. Dry Run Example

Dry run the template on Maximum XOR of Two Numbers in an Array: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'XOR trie' and asks for walk opposite bits first to maximize xor at each bit position., reach for XOR Trie before designing from scratch.

9. Common Mistakes

- Not marking word endings.
- Using char arithmetic that only works for lowercase when input is broader.
- Forgetting signed bit behavior.

10. Related LeetCode Problems

LC 421 Maximum XOR of Two Numbers in an Array

Family: Bit Manipulation

Pattern 90: Bit Manipulation

1. Pattern Name

90. Bit Manipulation

2. Signal (when to recognize this pattern)

Use XOR and bit masks for parity, uniqueness, missing numbers, and powers of two.

3. Keywords

XOR, single number, missing, bit, power of 2

4. Time Complexity

O(n)

5. Space Complexity

O(1)

6. Java 17 Template

Brute force: test every candidate directly.

Optimized approach: use bit operations to encode cancellation or membership in constant extra space.

The algorithm works because XOR and bit masks follow algebraic identities such as x ^ x = 0 and n & (n - 1) clearing one set bit.

Edge cases: zero, negative numbers for bit counts, duplicate assumptions, and integer width.

Java notes: Java int is signed 32-bit; use >>> for unsigned shifts and parentheses around bit expressions.

```
public int singleNumber(int[] nums) {
    int xor = 0; // XOR accumulator.
    for (int num : nums) xor ^= num; // Pairs cancel because x ^ x == 0.
    return xor; // Unique value remains.
}

public int missingNumber(int[] nums) {
    int xor = nums.length; // Include n.
    for (int i = 0; i < nums.length; i++) xor ^= i ^ nums[i]; // Cancel matching indices and values.
    return xor; // Missing value remains.
}

public int countSetBits(int n) {
    int count = 0; // Number of one bits.
    while (n != 0) { n &= n - 1; count++; } // Remove lowest set bit each loop.
    return count; // Total set bits.
}

public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0; // Powers of two have exactly one set bit.
}
```

7. Dry Run Example

Dry run the template on Single Number / Missing Number: initialize the state shown in the code, process the input in the loop order, write down every pointer/queue/stack/DP update, and update the answer exactly where the comment says the candidate is valid. The first time the maintained invariant produces a complete candidate, compare it with the current best.

8. Interview Recognition Trick

If the prompt says 'XOR' and asks for use xor and bit masks for parity, uniqueness, missing numbers, and powers of two., reach for Bit Manipulation before designing from scratch.

9. Common Mistakes

- Using logical operators instead of bitwise operators.
- Forgetting parentheses around n & (n - 1).
- Assuming bit tricks work with invalid duplicate counts.

10. Related LeetCode Problems

LC 136 Single Number; LC 268 Missing Number; LC 191 Number of 1 Bits; LC 231 Power of Two